

Transformer Deployment on Heterogeneous Many-Core Systems

Master Thesis

Author(s):

Wiese, Philip 

Publication date:

2023-05

Permanent link:

<https://doi.org/https://doi.org/10.3929/ethz-b-000714940>

Rights / license:

In Copyright - Non-Commercial Use Permitted

DEPARTMENT OF INFORMATION TECHNOLOGY AND
ELECTRICAL ENGINEERING

Autumn Semester 2022

Transformer Deployment on Heterogeneous Many-Core Systems

Master Thesis



Philip Wiese

wiesep@student.ethz.ch

May 2023

Supervisors: Moritz Scherer, scheremo@iis.ee.ethz.ch
Victor Jung, jungvi@iis.ee.ethz.ch
Gamze Islamoglu, gislamoglu@iis.ee.ethz.ch
Gianna Paulin, pauling@iis.ee.ethz.ch
Viviane Potocnik, vivianep@iis.ee.ethz.ch
Samuel Riedel, sriedel@iis.ee.ethz.ch

Professor: Prof. Dr. L. Benini, lbenini@iis.ee.ethz.ch

Acknowledgements

I would like to express my sincere gratitude to the Integrated Systems Laboratory (IIS) along with Prof. Benini for the exceptional opportunity and guidance provided throughout this thesis. I am also deeply grateful to my advisors Moritz, Victor, Gamze, Gianna, Viviane, and Samuel for their great support and invaluable insights throughout this project. Their mentorship has enabled me to develop new skills, work on a fascinating subject, and grow as a researcher and engineer.

Further, I would like to extend my earnest thanks to my dear friends Luca and Micheal for the stimulating discussions that fueled significant progress during the project. Moreover, I wish to thank my family and all my friends for their unrelenting encouragement and support throughout this journey. Their unwavering faith in me has been a constant source of motivation and inspiration.

Zurich, Mai 2023

Philip Wiese

Abstract

With the widespread application of neural networks across various domains, their ability to learn complex patterns from data and make accurate predictions has become increasingly valuable. As the utilization of machine learning on embedded systems continues to expand in different industries, there is a growing demand for reliable and efficient deployment frameworks that support heterogeneous many-core systems. In this work, we present an end-to-end deployment workflow that facilitates the seamless deployment of neural networks on heterogeneous many-cores systems that feature multiple user-managed memory hierarchies and hardware accelerators. To demonstrate the capabilities of our toolchain, we integrate MemPool with the Integer Transformer Accelerator (ITA) in our workflow composed of QuantLab and Deeploy and deploy a Transformer network based on the compact convolutional transformers (CCT) architecture. By leveraging ITA in MemPool, we achieve significant performance gains of up to $15\times$, energy efficiency improvements of $36\times$, and area efficiency gains of up to $74\times$.

Declaration of Originality

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor. For a detailed version of the declaration of originality, please refer to Appendix B.

Philip Wiese,
Zurich, May 2023

Relevant Repositories

In the following, a list of all the relevant code repositories used during the project is provided. Although most of the development efforts were performed on forked repositories, it is expected that the majority of changes will eventually be merged into the upstream repository.

- **Deeploy**

- Relevant: <https://iis-git.ee.ethz.ch/wiesep/deeploy>
- Upstream: <https://iis-git.ee.ethz.ch/deeploy/deeploy>

- **QuantLab**

- Relevant: <https://iis-git.ee.ethz.ch/wiesep/quantlab>
- Upstream: <https://iis-git.ee.ethz.ch/spmatteo/quantlab>

- **MemPool**

- Relevant: <https://iis-git.ee.ethz.ch/mempool/mempool>
- Upstream: <https://github.com/pulp-platform/mempool>

- **Snitch**

- Relevant: <https://github.com/Xeratec/snitch>
- Upstream: <https://github.com/pulp-platform/snitch>

- **Others**

- <https://iis-git.ee.ethz.ch/wiesep/deployitatests>

Contents

1. Introduction	1
2. Background	3
2.1. Neural Networks	3
2.1.1. Transformer Networks	4
2.1.2. Quantization	5
2.1.3. QuantLab	6
2.2. Computational Graphs	7
2.3. MemPool System	7
2.3.1. Integer Transformer Accelerator	8
2.3.2. Banshee	9
3. Related Work	10
3.1. STM32Cube.AI	10
3.2. TensorFlow Lite Micro	11
3.3. GWT GAPflow	11
3.4. DORY	12
4. Architecture	13
4.1. Overview	13
4.2. Deeploy	14
4.2.1. Under the Hood	15
5. Implementation	18
5.1. Isomorphic Graph Matching	18
5.1.1. Algorithm and Implementation	20
5.2. Modifications to Deeploy	20
5.2.1. Platforms	20
5.2.2. Passes	22
5.2.3. Kernels	22

Contents

5.2.4. Limitations	24
5.3. ITA Integration	26
5.3.1. Limitations	27
5.4. Other Contributions	28
6. Results and Evaluation	29
6.1. End-to-End Multi-Head Self-Attention	29
6.1.1. Performance	31
6.1.2. Efficiency	34
6.2. End-to-End Transformer Network	36
6.2.1. Runtime	37
6.3. Comparison with State of the Art	38
6.3.1. Attention Operation	38
6.3.2. Deploy	39
7. Conclusion and Future Work	41
7.1. Future Work	42
A. Task Description	43
B. Declaration of Originality	48
C. Source Code	50
C.1. Graph Matching Python Code	50
D. MemPool Memory Layout	54
E. Transformer Network	57
Bibliography	60

List of Figures

2.1. Transformer Architecture with Attention Mechanism	5
2.2. MemPool Architecture Overview	8
2.3. MemPool with ITA Architecture Overview	9
4.1. Deeploy Architecture Overview	16
4.2. Deeploy Backend Flow	17
5.1. Graph Matching Example	19
5.2. Matrix Multiplication Cycle Breakdown and Optimization	23
5.3. Attention Layer With and Without Explicit Requantization	24
5.4. Cycle Breakdown of a Transformer Network	25
5.5. Stall Breakdown of a Transformer Network	26
5.6. Quantized Softmax	27
6.1. Attention Network Graph	30
6.2. Deployed Attention Networks	32
6.3. MHSA Performance	33
6.4. ICCT Network Architecture	36
6.5. ICCT Runtime	37
D.1. MemPool Memory Layout with ITA	55
D.2. MemPool Memory Addressing	56
D.3. ITA Memory Layout	56
E.1. ICCT Network Graph	58

List of Tables

6.1. Attention Network Parameter	29
6.2. Attention Layer Performance and Efficiency Comparison	35
6.3. ICCT Network Parameter	37
6.4. Comparison to State-of-the-Art Transformer Accelerators	38
6.5. Comparison to State-of-the-Art GPUs	39
6.6. Comparison of Deeploy with State-of-the-Art Deployment Frameworks . .	40

List of Acronyms

AXI	Advanced eXtensible Interface
CCT	Compact Convolutional Transformers
CI	Continuous Integration
CIM	Compute-In-Memory
CNN	Convolutional Neural Network
DMA	Direct Memory Access
DNN	Deep Neural Network
DSP	Digital Signal Processing
FP	Full-Precision
FQ	Fake-Quantized
GE	Gate Equivalent
GPU	Graphics Processing Unit
IIS	Integrated Systems Laboratory
IoT	Internet of Things
IPU	Integer Processing Unit
ISA	Instruction Set Architecture
ITA	Integer Transformer Accelerator

List of Acronyms

LSTM	Long Short-Term Memory
MAC	Multiply-Accumulate
MHSA	Multi-Head Self Attention
NLP	Natural Language Processing
ONNX	Open Neural Network Exchange
PACT	Parameterized Clipping Activation
PTQ	Post-Training Quantization
PULP	Parallel Ultra Low Power
QAT	Quantization-Aware Training
RNN	Recurrent Neural Network
ROM	Read Only Memory
SGD	Stochastic-Gradient Descent
SHSA	Single-Head Self-Attention
SIMD	Single Instruction Multiple Data
SoC	System-on-Chip
SRAM	Static Random-Access Memory
TQ	True-Quantized
TQT	Trained Quantization Thresholds

Chapter 1

Introduction

Neural networks, particularly deep neural networks (DNNs) with many layers, have gained widespread application across various fields. They possess the ability to learn complex patterns in data and make accurate predictions. Some well-known and impressive examples are ChatGPT¹, AlphaFold² and DALL-E³. In the last few years, a new class of deep learning algorithm, the Transformer architecture [1], has emerged. Originally developed for natural language processing (NLP), Transformers are nowadays successfully applied to a wide range of problems such as text generation [2], text classification [3], image classification [4] and image generation [5], as well as audio processing [6]. However, the attention mechanism in Transformer networks imposes a high computational load and a complex operational scheme. To address this challenge, Islamoglu et al. [7] developed the Integer Transformer Accelerator (ITA) for the MemPool many-core system to accelerate the attention operation in Transformer networks.

Additionally, as the use of machine learning on embedded systems, often referred to as TinyML, continues to grow in various industries, there is an increased need for reliable and efficient deployment frameworks [8]. Deployment involves transforming the neural network into an optimized, platform-specific representation and generating the necessary code to execute the model efficiently on the selected platform. In response to this surging demand, a number of frameworks for deploying neural networks on embedded systems have emerged in recent years, reflecting the increasing importance of developing robust and effective deployment solutions. However, developing such a framework is not a trivial task and requires careful consideration of various factors such as compatibility with different platforms, performance, memory usage, power consumption, extensibility, and ease of use. Considering the trend for increased heterogeneity of the hardware with many compute cores in combination with specialized accelerators, deployment often

¹OpenAI ChatGPT <https://openai.com/blog/chatgpt>

²DeepMind Alpha Fold <https://www.deepmind.com/research/highlighted-research/alphafold>

³OpenAI DALL-E <https://openai.com/research/dall-e>

1. Introduction

involves complex mapping of the computational graph, representing the neural network, to the hardware. The use of heterogeneous systems allows for improved energy efficiency, scalability, and data throughput. However, especially for embedded devices, this also comes with some difficulties such as strict memory constraints, the need for explicit memory transfers with user-managed scratchpad memory, and interaction between all components of the system.

Addressing these challenges, the objective of this master thesis is to develop an end-to-end deployment process that facilitates the seamless deployment of neural networks, with a focus on Transformers, on embedded systems with our new deployment tool called Deeploy. This involves all steps from the architecture design and training of the network to running the quantized model on specialized hardware, in a semi-automated and streamlined procedure. In detail, we make the following contributions:

- We provide a pipeline to deploy neural networks on heterogeneous many-core systems. This includes quantization, automatic hardware-aware graph optimization, and optimized C code generation.
- We demonstrate the end-to-end deployment of a quantized compact convolutional transformers (CCT) network on the MemPool platform with the Integer Transformer Accelerator (ITA).
- We evaluate the performance of the attention operation present in Transformer networks on MemPool, both with and without ITA.

This thesis is structured as follows: Chapter 2 introduces all relevant concepts and additional background information including Transformer networks, quantization, computational graphs, and the MemPool system. In Chapter 3, some of the most popular and capable deployment frameworks are introduced and their respective pros and cons are highlighted. Next, in Chapter 4, the general idea behind neural network deployment and the architecture as well as the core principles of Deeploy are discussed. Chapter 5 in detail explains the implementation with a focus on graph matching, integration of ITA, and modifications made in Deeploy. This is followed by Chapter 6, which presents the performance analysis of the attention operation on MemPool, both with and without ITA. The evaluation includes a comparison to state-of-the-art Transformer accelerators and the latest NVIDIA graphics processing units (GPUs). Additionally, the chapter evaluates the runtime of a complete Transformer network and compares the functionalities of Deeploy with other frameworks. Finally, Chapter 7 summarizes the project, draws a conclusion, and provides some ideas for future work.

Chapter 2

Background

This chapter introduces all relevant concepts and is structured as follows: Section 2.1 provides relevant background information about neural networks, specifically Transformers. Additionally, it briefly explains quantization and introduces QuantLab, a software framework designed for the training and quantization of neural networks. Section 2.2 introduces computational graphs, as the deployment of neural networks often involves the transformation and translation of such data structures. Finally, Section 2.3 contains information about the MemPool platform, which was selected to demonstrate the end-to-end deployment developed in this thesis.

2.1. Neural Networks

Neural networks are a type of machine learning algorithm designed to learn patterns and make predictions based from new data. These networks are composed of multiple layers of artificial neurons, each layer processing the output from the previous layer. The weighted inputs of a neuron are fed through an activation function, such as ReLU or softmax, to introduce non-linearity and enhance the network's ability to learn complex patterns. The process of training a neural network involves employing a loss function that quantifies the disparity between the network's predicted output and the desired output. By calculating the gradients of the loss function with respect to the weights, stochastic gradient descent (SGD) can be used to adjust the network parameters in a manner that reduces the loss. However, because the loss function is typically non-convex, training a neural network with SGD may not converge to the global optimum, but rather to a local minimum.

2. Background

2.1.1. Transformer Networks

Introduced in 2017 by Vaswani et al. [1], Transformers are a neural network architecture originally developed for NLP. The main novelty of Transformer networks is the use of the attention mechanism, which allows the network to attend to different parts of the input sequence during training and inference. This is especially valuable when working with long-sequence data and capturing complex patterns. Transformers have become popular in recent years due to their ability to achieve state-of-the-art performance on a wide range of applications. In the original implementation [1], a Transformer network consists of L encoder and M decoder layers, as shown in Figure 2.1. Nowadays, architectures with only decoder or encoder modules are also often used.

The dimension of the layers in the multi-head attention are defined as sequence length (S), embedding size (E), projection space (P), and the number of heads (H). In related work, the projection space P is often defined as $P = d_{model}/H$ where d_{model} is the model dimension. The attention layer takes three inputs: queries ($\mathbf{q}$), keys ($\mathbf{k}$), and values ($\mathbf{v}$), and maps a query and a set of key-value pairs to an output (O) [1]. These inputs have dimension $S \times E$ and are first linearly projected to H heads, each with shape $S \times P$. Next, the attention matrix is calculated as:

$$\mathbf{A} = \text{softmax} \left(\frac{\mathbf{QK}^T}{\sqrt{P}} \right) \quad (2.1)$$

The matrix $\mathbf{A}$ with dimension $S \times S$ can be considered as probabilities representing the relationship between queries and values. Finally, $\mathbf{A}$ is multiplied by $\mathbf{V}$, and the output of all heads is concatenated and linearly transformed to match the dimension of the input ($S \times E$). In summary, the output is computed as a weighted sum of the values with the weights given by the attention matrix between queries and keys¹. In the case of multi-head self-attention (MHSA), key, queries, and values are identical.

However, one downside of the Transformer architecture is its high computation and memory demands. This comes mainly from the attention mechanism itself, illustrated on the right side of Figure 2.1, which requires pair-wise computation between all elements of the input sequence. Thus, the number of operations scales quadratically with the sequence length and features a low data reuse potential. To mitigate the computational challenges, researchers have developed methods to reduce the computational cost while maintaining their performance [9]. Still, the use of Transformer networks with long input sequences on embedded platforms with memory and power constraints is not trivial. The demand to efficiently use said networks has led to the development of specialized hardware such as the MemPool platform, with ITA, which is further discussed in Section 2.3.

¹For a more in detail explanation, refer to <https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452>.

2. Background

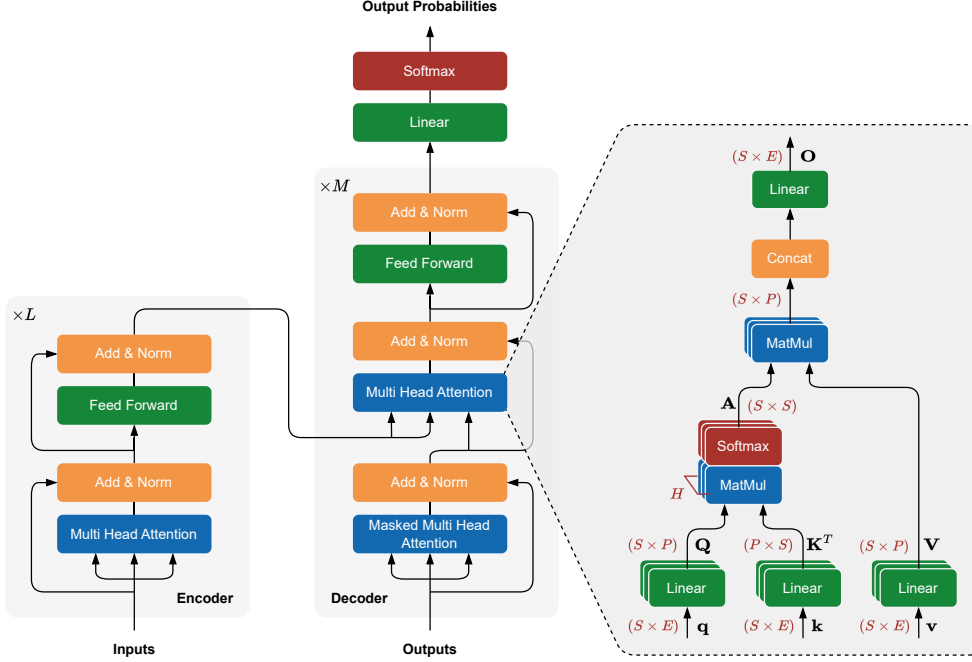


Figure 2.1.: Transformer architecture with attention mechanism, proposed by [1]. The output of the attention is computed as a weighted sum of the values (**v**) with the weights given by the attention matrix (**A**) between queries (**q**) and keys (**k**).

2.1.2. Quantization

Quantization is a technique that compresses the implementation cost of deep learning models by representing the weights and activations with a lower number of bits. This allows for faster and more energy-efficient model inference at the cost of reduced accuracy. In general, quantization can be achieved in two ways: post-training quantization (PTQ) and quantization-aware training (QAT) [10]. The former is a technique where the weights and activations of a pre-trained neural network are quantized after the training process is complete. This involves converting the floating-point values to integer values and optimizing the quantized model to minimize the loss in accuracy caused by quantization. Alternatively, QAT is a process where a model is trained with the intention of being quantized afterward. During training, the model is exposed to quantization effects by using lower-precision values for weights and activations, which enables the model to learn how to perform well under quantization constraints. More information about quantization as part of the deployment process is provided in Chapter 4.

In comparison, PTQ is typically much faster than QAT because the weights and activations are fixed, and the quantization process can be done offline. However, the loss in accuracy can be significant, and fine-tuning the quantized model may be required to

2. Background

recover some of the lost accuracy. On the other hand, QAT is more accurate but slower as it involves training the network to become robust to quantization during the training process. In practice, various different algorithms for the quantization of neural networks are used in industry and academia [11, 12, 13, 14].

Uniform quantization is a process of mapping a real number x to a discrete quantized representation $x_q \in [-2^{B-1}, 2^{B-1} - 1]$. The quantum of each level ϵ , also referred to as the scaling factor, is determined by the bit precision B and the clipping parameters α and β . The quantization function Q is defined as

$$x_q = Q(x, B, \alpha, \beta) = \left\lfloor \frac{\text{clip}(x, \alpha, \beta)}{\epsilon} \right\rfloor \quad (2.2)$$

$$\epsilon = \frac{\beta - \alpha}{2^B - 1} \quad (2.3)$$

To dequantize the transformed values back to the real domain, the de-quantization operator can be used, defined as:

$$\tilde{x} = \text{DQ}(x_q, \epsilon) = \epsilon x_q \approx x \quad (2.4)$$

In addition, symmetric quantization is a specific type of quantization, where the clipping parameters fulfill $\alpha = -\beta$, with the advantage that no bias term needs to be considered during matrix multiplication and convolution operations [14].

2.1.3. QuantLab

QuantLab is a versatile and modular framework designed for training, comparing, and quantizing neural networks [15]. The framework is based on PyTorch² and supports sub-byte and mixed precision quantization. Furthermore, QuantLib is a component of QuantLab containing all quantization-related code. QuantLib already includes several QAT algorithms, such as Parameterized Clipping Activation (PACT) [13] and Trained Quantization Thresholds (TQT) [14]. The architecture facilitates straightforward integration of new quantization algorithms and modules, allowing for easy transformation of a network from a floating point to a true quantized representation. This also enables the integration of hardware-aware quantization strategies, which are necessary when utilizing specialized machine-learning accelerators that comprise custom approximation modules, ultimately leading to a significant increase in energy efficiency.

In addition, QuantLib incorporates epsilon, signedness, and precision propagation. The former is essential for non-linear functions and for layers where inputs with possibly different scaling factors are combined. Consequently, QuantLab inserts re-quantization layers before all inputs to harmonize the scaling factors. Precision propagation, respectively

²PyTorch is a free and open-source machine learning framework available under <https://pytorch.org/>

2. Background

number of levels propagation, enables quantizing mixed precision networks, achieving a balance between computational efficiency, memory requirements, and energy consumption.

2.2. Computational Graphs

In general, a computational graph is a directed acyclic graph that represents the flow of computation in an algorithm. Hereby, each node in the graph represents a specific operation or computation, and the edges represent the data dependencies between these operations. Each node takes inputs from one or multiple incoming edges, performs a computation, and produces an output that is passed to its outgoing edges. In summary, the graph structure defines the order in which the operations are performed and the dependencies between them.

Computational graphs are a powerful tool for representing and optimizing the flow of data in neural networks. Thereby, nodes represent mathematical operations such as matrix multiplication or activation functions, making this representation especially helpful in the deployment of machine learning models. By allowing for hardware-agnostic and hardware-aware optimization of the mathematical structure of the network, computational graphs facilitate the efficient and effective deployment of these networks toward various goals. Furthermore, they enable portability across different hardware and software platforms, such as the Open Neural Network Exchange (ONNX)³ format, and enable automated scheduling and tiling algorithms to improve the execution of computations on parallel and memory-constraint hardware.

2.3. MemPool System

The MemPool platform [16], developed by the IIS at ETH Zurich, is a flexible and parametric many-core cluster based on the RISC-V instruction set architecture (ISA) [16]. At the heart of the system in its default configuration are 256 RV32IMAXpulp⁴ *Snitch* cores, which are arranged hierarchically to function as processing engines, along with a scalable distributed 1 MiB shared L1 scratchpad memory that can be accessed within one to five cycles in the absence of conflicts.

The system is organized into four hierarchical levels - *tile*, *group*, *cluster*, and *system*. Each *tile* comprises four *Snitch* cores and 16 kB of static random-access memory (SRAM), divided into 16 banks. A *group* is formed by combining 16 *tiles* with local and remote interconnects in between *tiles*, as illustrated in Figure 2.2. The *cluster* comprises four

³Open Neural Network Exchange Homepage: <https://onnx.ai/>

⁴RISC-V 32-bit base instruction set with additional support for integer multiplication and division (M), atomic instructions (A), and a custom extension targeting digital signal processing (DSP) applications [17].

2. Background

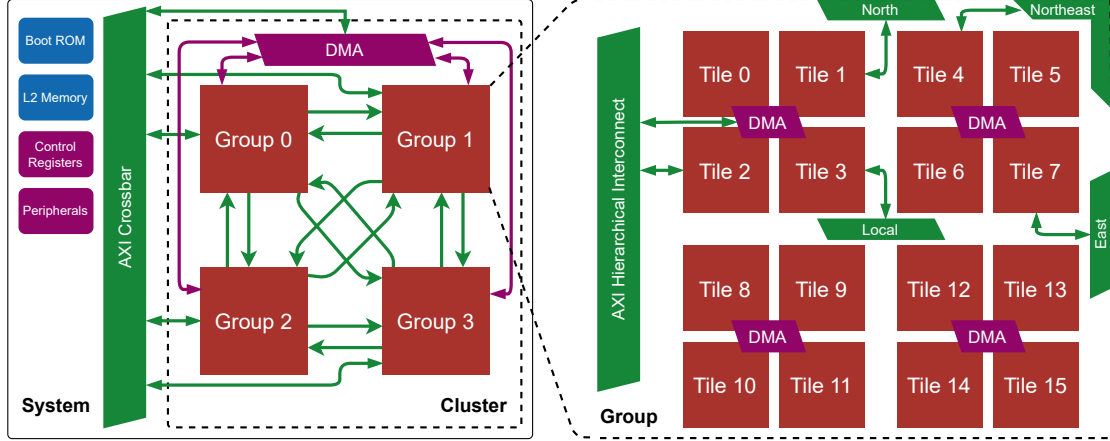


Figure 2.2.: Overview of MemPool’s architecture, detailing its hierarchy and interconnects. Each *tile* consists of four cores with 16 banks of L1 scratchpad memory. A *group* holds 16 *tiles* with local, remote, and Advanced eXtensible Interface (AXI) interconnects highlighted in green. Not all connections in the *group* are shown, but each *tile* and DMA is connected to all interconnects. The *cluster* features four *groups* and is connected to additional memory and peripherals of the *SoC* through the AXI crossbar.

groups and includes a hierarchical AXI interconnect to access lower memory levels, the boot read-only memory (ROM), the *cluster* direct memory access (DMA) engine, and the control registers of the *System-on-Chip* (*SoC*).

2.3.1. Integer Transformer Accelerator

Hardware accelerators can significantly speed up the computations involved in the attention mechanisms, which typically involve large matrix multiplications, complex dataflow dependencies, and large memory requirements. This can result in improved throughput, reduced latency, and significantly improves the energy efficiency of the system. One such hardware accelerator is the Integer Transformer Accelerator (ITA) focusing on 8-bit quantized Transformer networks and accelerating the attention mechanics.

Additionally, it employs a novel and hardware-friendly softmax approximation that works directly on the quantized values and does not require costly exponentiation or multiplication. The implementation proposed by Islamoglu et al. [7] features computation on streaming data, thus avoiding fetching the same vector multiple times, reducing data movements and power consumption. Consequently, the implementation features partial softmax computation, as the $\mathbf{QK}^T$ operation generates tiles of size $M \times N$. As soon as all rows of A are accumulated in the sum buffer, $\mathbf{AV}$ is computed while simultaneously normalizing the elements of A in the softmax module. Most important, the softmax computation does not add any latency. More details are provided in [18] and [7].

2. Background

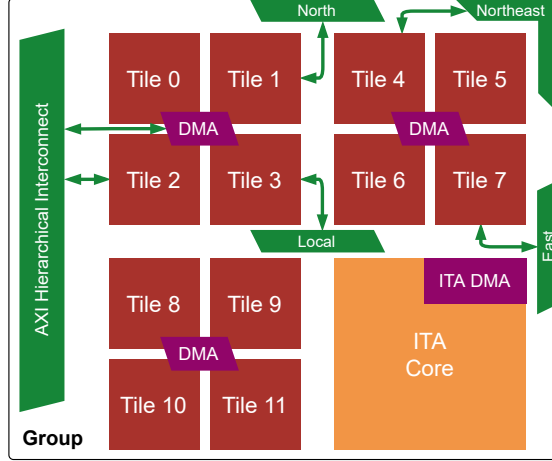


Figure 2.3.: Overview of a MemPool *group* with ITA. Four *tiles* are replaced with an ITA core featuring a custom DMA. One ITA core operates on 64 memory banks, resulting in 64 kB of SRAM. Not all connections in the *group* are shown, but each *tile*, DMA, and ITA core is connected to all interconnects.

In this work, we utilized a variant of the MemPool system, wherein four out of sixteen *tiles* of a *group* were substituted with an ITA core, as illustrated in Figure 2.3. One ITA core features $N = 16$ processing units, each consisting of $M = 64$ multiply-accumulate (MAC) units. Although the 64 memory banks in the original *tiles* are still accessible to MemPool, they are exclusively assigned to the ITA core during computations. Consequently, the MemPool system holds four ITA cores in total, with each one featuring its custom DMA, facilitating non-continuous data access during computation. Relevant information about the memory map of MemPool with ITA and the layout of the memory assigned to the accelerator can be found in Appendix D.

2.3.2. Banshee

Banshee is a fast binary simulator supporting up to 618 MIPS⁵ for single-core execution by using static binary translation based on the LLVM infrastructure [19]. The significant advantage of Banshee is its capacity to simulate hundreds of RISC-V cores concurrently. While hardware simulators Siemens QuestSim[®] or Synopsys VCS[®] are cycle-accurate, Banshee enables fast development by generating instruction-accurate simulations that are orders of magnitude faster than cycle-accurate simulations. This is especially valuable when working with complex systems such as MemPool.

⁵Million instructions per second

Chapter 3

Related Work

This chapter provides a brief overview of some of the most popular and relevant frameworks for this work and highlights their respective advantages and disadvantages. It is important to note that the following list is not exhaustive but is intended to provide an informative overview of some of the most prominent and relevant frameworks.

3.1. STM32Cube.AI

STM32Cube.AI¹ is a machine learning deployment tool developed by STMicroelectronics (STM) targeting their 32-bit ARM-based microcontrollers. The tool provides a user-friendly interface to configure and optimize the neural network for the target hardware. The software tool supports several types of neural networks, including convolutional neural networks (CNNs), recurrent neural networks (RNNs), and long short-term memory (LSTM) networks, and it is compatible with various popular machine learning frameworks such as TensorFlow Lite, Keras, qKeras, PyTorch, and ONNX. Moreover, STM32Cube.AI features direct C code generation and leverages fast, optimized open-source CMSIS²-NN kernels.

However, it is exclusively designed for use with STM microcontrollers and cannot be used with other platforms. Additionally, the source code of the tool itself is not open-source, and there is no support for integrating custom operators. This limitation restricts its suitability for research and security applications. Moreover, integrating third-party software with the tool is more complex compared to the open-source alternatives.

¹More information under https://www.st.com/content/st_com/en/campaigns/stm32cube-ai.html

²Common Microcontroller Software Interface Standard (CMSIS) is a vendor-independent abstraction layer for microcontrollers that are based on ARM Cortex processors.

3.2. TensorFlow Lite Micro

TensorFlow Lite Micro (TFLM)³ is an open-source machine learning framework developed by Google Brain and released under the Apache 2.0 license in 2019. It is specifically designed for microcontroller-class devices with limited resources, such as memory and processing power. TFLM is an extension of the TensorFlow Lite framework, and it supports different hardware platforms, including microcontrollers, edge devices, and smartphones. It has gained popularity over the past years and has a large community of contributors.

Although TFLM theoretically supports the addition of custom operators, implementing them can be challenging and require extensive modifications. Furthermore, TFLM's interpreter-based approach to network inference may result in lower performance compared to other solutions. One of the main limitations of TFLM is, that it only supports single-core processors and has no support for user-managed multi-level memory architectures requiring memory tiling, nor does it support hardware accelerators or multicore systems.

3.3. GWT GAPflow

GAPflow⁴ is a set of tools developed by GreenWaves Technologies to facilitate the porting of neural networks to their Internet of Things (IoT) processors. It consists of the open-source GAP NNtool and the closed-source GAP Autotiler and provides end-to-end deployment, including quantization and model calibration. The tool supports custom operators and matrix tiling with multiple user-managed memory levels, and it generates C code optimized for the GAP processors.

While GAPflow is a powerful tool for optimizing neural networks for embedded systems, it has quite some limitations. Currently, it only supports the GAP8 and GAP9 processors, and only works with TensorFlow Lite models. Additionally, the tool has a steep learning curve and requires in-depth expertise in machine learning and embedded systems. Furthermore, the constraint-based tiler is proprietary and therefore lacks transparency. Since the underlying algorithms and decision-making processes are not openly available, it becomes challenging to understand the inner workings of the tiling process. Consequently, if the proposed tiling turns out to be invalid or requires modifications, the only available option for resolution is to seek assistance from GreenWaves's engineers. Finally, the documentation and community support are also limited.

³More information under <https://www.tensorflow.org/lite/microcontrollers>

⁴More information under <https://greenwaves-technologies.com/gapflow/>

3.4. DORY

DORY⁵ is an end-to-end deployment tool that automates the process of optimizing and deploying neural networks on embedded systems [20]. It integrates data and loop tiling and avoids memory fragmentation to generate efficient C code for the parallel ultra low power (PULP) platform. DORY supports quantized ONNX models exported with the open-source NEMO⁶ tool or the newer QuantLab⁷ framework, described in Section 2.1.3. Further, it leverages fast and optimized open-source PULP-NN kernels and features direct C code generation.

However, the tool has limited platform support, currently supporting only GAP8 and partially supporting Diana [21], a digital and analog hybrid neural network SoC developed at KU Leuven, as well as Occamy, a dual chiplet system with more than 400 RISC-V cores, developed at IIS at ETH Zurich. While it is possible to add support for new platforms, this requires significant effort and requires in-depth knowledge of the software architecture. Additionally, only a limited set of layer types is currently supported and customization of the framework can be challenging. Furthermore, the tool only supports up to three concurrent branches in the network.

⁵More information under <https://github.com/pulp-platform/dory>

⁶More information under <https://github.com/pulp-platform/nemo>

⁷More information under <https://github.com/pulp-platform/quantlab>

Chapter 4

Architecture

This chapter provides a brief overview of the general idea behind end-to-end neural network deployment. Additionally, the underlying architecture of Deeploy is explained in Section 4.2.

The initial architecture and preliminary implementation of Deeploy were provided by Moritz Scherer at the start of this project. Subsequently, during the course of this thesis, additional features were integrated, and the existing codebase was restructured. Chapter 5 delves into these modifications in greater depth.

4.1. Overview

The objective of this master thesis is to develop an end-to-end deployment process that facilitates the seamless deployment of neural networks on embedded systems. This involves all steps from the development stage to running the network on specialized hardware, in a semi-automated and streamlined procedure. In general, this procedure can be categorized into three components – *frontend*, *middleware*, and *backend*.

The *frontend* handles the training and quantization of the neural network. During the training process, a full-precision (FP) model is optimized by adjusting the weights and biases through the use of backpropagation, evaluating its performance, and performing hyperparameter tuning. The quantization process involves converting the model into a fake-quantized (FQ) representation, where the number of possible values for weights, biases, and activations is restricted to a finite set to emulate the effect of quantization. This conversion can be done using either PTQ or QAT, as explained in Section 2.1.2. Finally, the model is transformed into a true-quantized (TQ) representation, where all parameters and operations are quantized to the integer domain.

4. Architecture

Next, the *middleware* is responsible for the intermediate representation and optimization of the network. The intermediate representation is an abstract representation of the neural network that is used to optimize the computational graph. The optimization process involves reducing the computational complexity of the model by simplifying the network architecture and transforming the graph to a deployable format. This step also includes the application of hardware-aware optimizations such as layer fusion, tensor layout transformation, and node splitting. This process improves performance, reduces the memory requirements of the model, and ensures that the resulting graph can be deployed on the target platform.

Finally, the *backend* is responsible for generating code for the target hardware platform. This includes layer scheduling, matrix tiling, strict type checking, and generating code for the kernels, which are small functions that perform the basic operations of the neural network, such as convolutions and matrix multiplications. The generated code is optimized for the target platform, and it is typically written in low-level languages such as C, C++, or Rust.

In summary, the *frontend* handles the training and quantization of the model, the *middleware* optimizes the computation graph, and the *backend* generates optimized code for the target hardware. By separating the deployment process into these three distinct components, the development and optimization of each component becomes easier. Further, modularity allows developers to reuse components for different models and across different hardware platforms.

4.2. Deeploy

Deeploy is a tool implementing the *middleware* and *backend* components of an end-to-end deployment toolchain. It's important to note that Deeploy is generally *frontend*-agnostic and consumes standard ONNX nodes. However, the parsers are currently targeted toward the export format of QuantLab, described previously in Section 2.1.3. The tool facilitates fully automated conversion, optimization, and deployment of ONNX graphs to C programmable platforms. At the time of writing, the supported *backend* platforms include ARM processors such as the STM32 series; Siracusa, an IoT SoC of the PULP family; as well as MemPool, described in Section 2.3.

The central component in Deeploy is the platform class, which serves as the primary interface for end users [22]. Its role is to furnish Deeploy with critical information about the target *backend* platform, such as:

- **The layer mapping**, that defines the relationship between ONNX nodes and Deeploy layers.
- **The node mapping**, which specifies the relationship between layers and the possible kernels that could be used.

4. Architecture

- **The optimizer**, comprised of a list of optimization passes applied in the *middleware*.
- **The storage objects**, including information on how to generate allocation and de-allocation code.

Therefore, implementing a new platform in Deeploy involves incorporating or adapting these four key concepts. Adding new kernels and features to Deeploy is also quite simple due to the modular structure, resulting in minimal additional implementation overhead. A more detailed overview of Deeploy’s architecture is provided in the next section.

4.2.1. Under the Hood

Disclaimer

Note that the information provided in this section is specific to the version of Deeploy at the time of writing, and likely is subject to change in future releases. Deeploy is constantly evolving and updates or modifications may alter the functionalities, features, or processes described here. It is recommended to consult the official repository for the most up-to-date information.

The core component is the Deployer shown in Figure 4.1, which performs several crucial steps. It requires the before-mentioned platform-dependent optimizer, mapping, storage objects, the ONNX graph, and a node scheduler that determines the execution order of the nodes.

The first step involves parsing the entire graph and conducting sanity checks to infer the type and signedness of global inputs and outputs. Next, the platform-specific optimizations are applied to transform the graph into a lowered, deployable structure. Subsequently, the lowered graph is subjected to additional post-lowering passes before being passed to the *backend*. These lowering passes may include, for instance, transposing constant inputs for matrix multiplication or converting from an NCHW¹ to an NHWC tensor layout to boost computational efficiency and match the requirements of the *backend* implementation and kernels themselves.

In the *backend*, each ONNX node is mapped to an internal layer class based on its *opcode*, reflecting the type of operation. Each layer is first parsed using a layer-specific context-free parser to ensure that all required attributes are present and that the specific combination is supported by Deeploy. Next, the layers are passed through the context-bound parser, which builds an internal representation of the layer operation, later used for code generation. This includes, for example, the input and output buffer names, shape information, and the node parameter. During this process, a single layer can be linked to multiple mappers, each of which can contain multiple bindings for different

¹Acronym to describe the order of axes of a tensor and often used in the context of image data. (N) Number of batches, (C) Number of channels, (H) Height, (W) Width.

4. Architecture

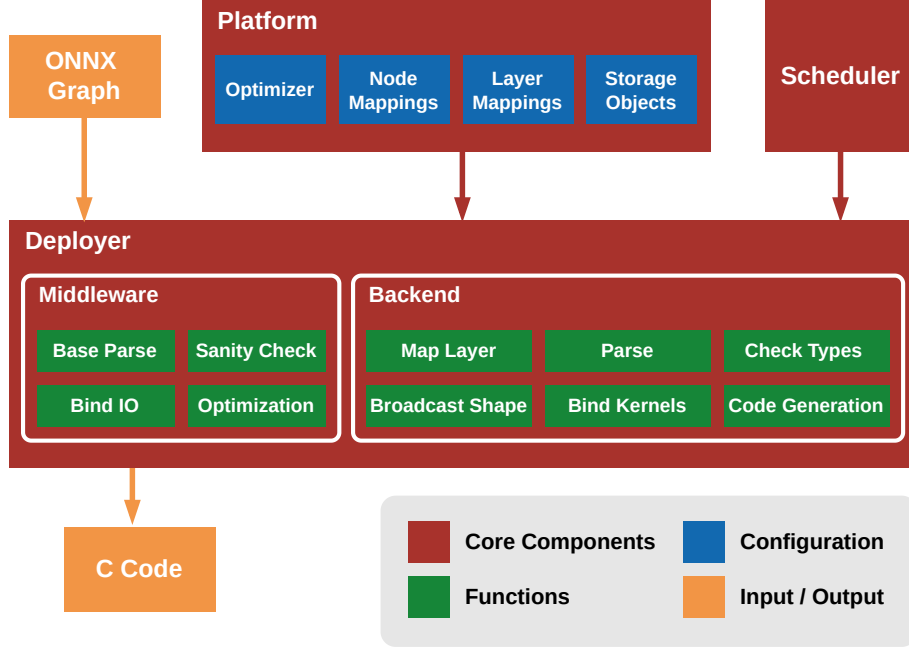


Figure 4.1.: Overview of Deeploy’s software architecture, functioning as *middleware* and *backend* in an end-to-end deployment workflow. Deeploy is *frontend*-agnostic and consumes standard ONNX graphs. After parsing, broadcasting, and optimization, each node is bound to a kernel template used for code generation.

kernels, as shown in Figure 4.2. The compatibility of a specific mapping is determined by the node parser. Hence, the first matching mapper is selected, linking together a set of possible bindings to a layer, which itself is mapped to an ONNX node in the lowered graph.

Following the layer mapping, the shapes of all tensors in the lowered graph are broadcasted, the layout information is propagated, and the number of operations for each node is calculated. Before binding the layers to a specific kernel function, the types of the inputs and outputs of each node are checked and updated, if necessary, to ensure that Deeploy uses the minimum number of bits required to represent a tensor. Similar to layer mapping, the first matching kernel binding is selected, where all inputs and outputs fulfill the data type constraints. This condition is ensured by the type checker assigned to each binding. After layer binding, the type of the output is updated to ensure that the subsequent layers have up-to-date information and that the complete graph is executable.

Finally, the C code is generated based on the templates bound to each node in the ONNX graph. Deeploy performs liveness analysis during code generation by tracking the state of each allocated buffer, allowing it to detect memory leaks and determine the maximum buffer allocation.

4. Architecture

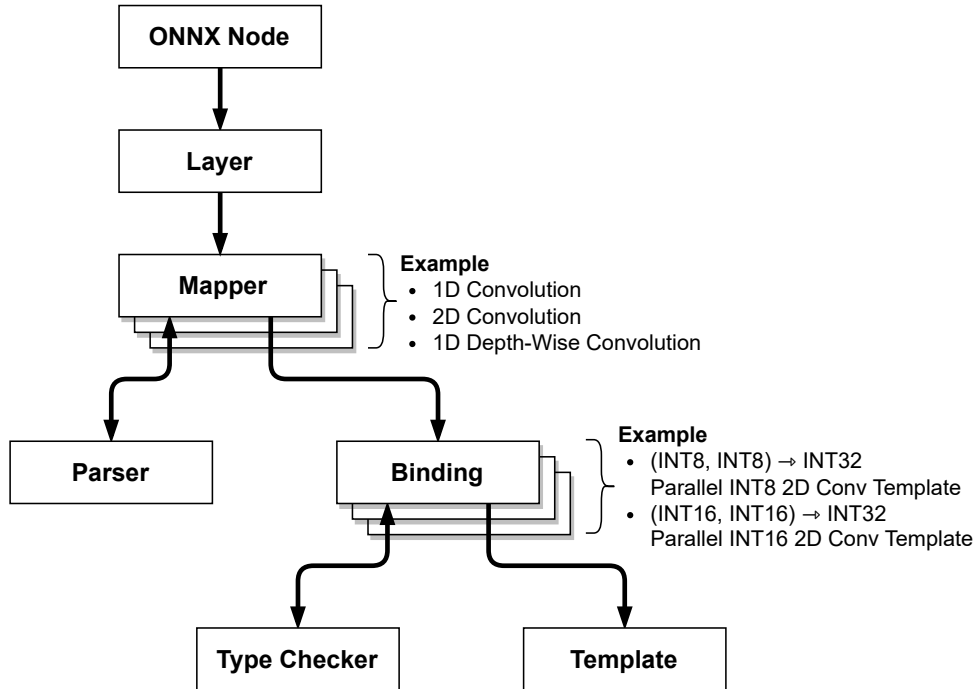


Figure 4.2.: Flowchart depicting Deeploy's backend flow. Each node in the graph is mapped to an internal layers class, each of which may contain several mappers. The compatibility of a specific mapping is determined by the node parser during the layer mapping stage. Similarly, each mapper can hold multiple potential bindings for various input and output types. During the layer binding step, following the shape and type inference, a layer is bound to a specific kernel template.

Chapter 5

Implementation

This chapter presents the final implementation to facilitate end-to-end deployment of a Transformer network on heterogeneous many-core systems. To demonstrate the full capability of our workflow, the MemPool platform was selected due to the integrated hardware accelerator to boost the performance of the attention mechanism in the Transformer architecture. During this project, we made several modifications and implemented additional features, which can broadly be categorized into hardware-agnostic and hardware-specific changes. The latter are in this case, specific to MemPool.

This chapter is structured as follows: The purpose and implementation of isomorphic graph matching and node fusion are explained in Section 5.1. Next, Section 5.2 provides detailed information on the new kernel functions and the current state of optimization. The integration of ITA into Deeploy and QuantLab is discussed in Section 5.3. Finally, other miscellaneous contributions are mentioned in Section 5.4.

5.1. Isomorphic Graph Matching

To optimize the computational efficiency, offload some operations to accelerators, or fulfill certain requirements, Deeploy must be able to match and extract given patterns from the computational graph. Specifically, the search is for one or more subgraph isomorphisms between a pattern graph and the network’s computational graph, subject to additional conditions on the leaf nodes. Once matched, these subgraphs can be transformed, provided the computational equivalence of the original subgraph is maintained. For instance, re-quantization nodes can be merged with matrix multiplication to reduce unnecessary memory operations and improve performance. Another example is splitting a convolution node with padding into two separate nodes. One dedicated padding node and another convolution node without padding, to support platforms lacking a padding-capable convolution kernel. Another use case is offloading computation to a dedicated

5. Implementation

accelerator, in which the node represents the interface between the processor and the accelerator.

A valid mapping between a pattern and the searched graph has to fulfill the following conditions for all nodes:

- (1) The color / operation is identical.
- (2) For input leaves of the pattern, the number of outputs is identical.
- (3) For output leaves of the pattern, the number of inputs is identical.
- (4) For intermediate nodes, the number of inputs and outputs is identical.

Output leaves in the pattern only have inputs, while input leaves only have outputs.

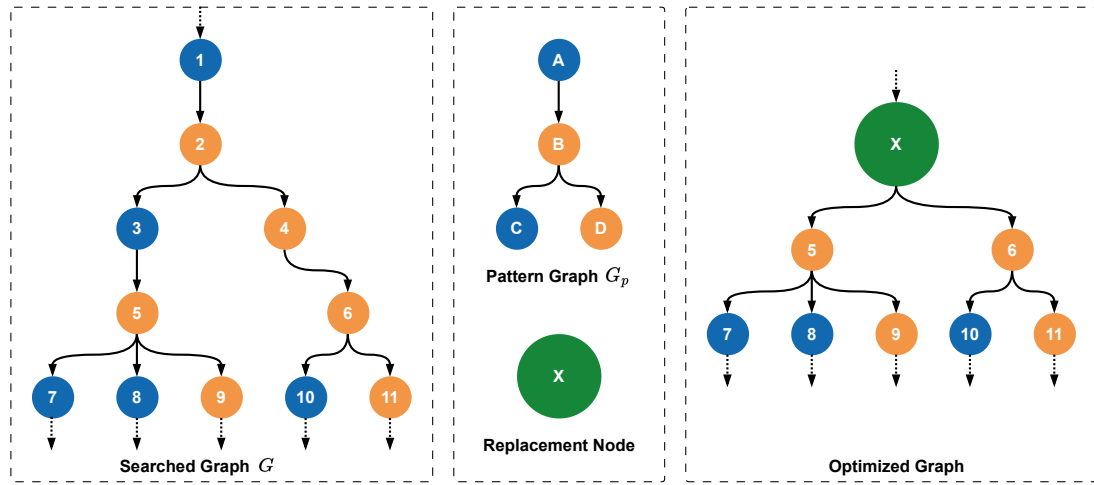


Figure 5.1.: Example illustrating the matching between a pattern graph G_p and the search graph G . The matched nodes are fused into the larger green node X .

In the example depicted in Figure 5.1, the only possible match is

$$M_1 = \{(A, 1), (B, 2), (C, 3), (D, 4)\}$$

and can thus be fused to a larger node shown in green. However, the mapping

$$M_2 = \{(A, 3), (B, 5), (C, 8), (D, 9)\}$$

is not a valid match because node B and node 5 do not have the same number of edges, thus violating condition (4). Similarly, the mapping

$$M_3 = \{(A, 4), (B, 6), (C, 10), (D, 11)\}$$

is not a valid match since node A and node 4 do not share the same color, violating condition (1).

5. Implementation

5.1.1. Algorithm and Implementation

The neural networks targeted by Deeploy can be represented as k -colored, directed, acyclic, and connected graphs G , with each color representing a unique operation or a constant value node. To match subgraphs, Algorithm 1 is proposed, which operates on a subgraph of G without constant nodes (Line 3). This is feasible because inputs with a constant value can be regarded as internal parameters of a node, thereby reducing the complexity of the pattern graph and simplifying the use of the pattern-matching functionality. However, the constant folding is not done automatically but has to be handled by the user to ensure that after the optimization pass, the resulting graph is computationally equivalent.

The pattern-matching process can start at any node on the pattern graph G_p , as indicated in the `MatchGraph` function in Line 1. In Line 10, if the current nodes in the pattern and search graph satisfy the aforementioned conditions, implemented in the `CompareNode` function in Line 7, the matcher can recursively attempt to match all parent (Line 12) and child (Line 21) nodes in the pattern with the searched graph G . A match is valid only if all nodes in the pattern are successfully paired with a node in G . This condition is checked in Lines 16, 23 and 27, causing the recursion to halt as soon as a pattern node cannot be matched to a node in G . The implementation in Python is provided in Appendix C.1.

5.2. Modifications to Deeploy

To facilitate end-to-end deployment of a neural network and particularly Transformer networks with Deeploy, additional kernels were implemented, existing ones optimized, and the list of supported platforms extended. Further, the overall code was refactored to improve future development and make it easier for multiple developers to contribute to the framework.

5.2.1. Platforms

To support new architectures, Deeploy can easily be extended with new platforms with their own specialized optimization passes and kernels. To streamline this process, the code base was restructured, and the existing testing code was extended to enhance continuous integration (CI) and automated testing of individual kernels and new features. Currently, Deeploy supports CMSIS, PULP, MemPool, and a generic platform for generating platform-agnostic C code that runs on any system. The latter two platforms were developed as part of this thesis. Additionally, to simplify the compilation process, a build workflow based on CMake was introduced.

5. Implementation

Algorithm 1: Isomorphic Graph Matching

Input : Searched graph $G = (V, E)$, k -colored, acyclic, connected

Pattern graph $G_p = (V_p, E_p)$, p -colored, acyclic, connected

Output: List of non-overlapping matches. A match $m : V_p \rightarrow V$ maps each node in G_p uniquely to a node in G .

```

1 Function MatchGraph( $G, G_p$ ):
    // We can start matching the pattern from any element in  $V_p$ 
2    $v_p \leftarrow v \in V_p$ ;
3    $V_s = \text{GetNonConstantNodes}(G)$ ;
4   matches  $\leftarrow []$ ;
5   foreach  $v_s$  in  $V_s$  do matches.add(MatchNodes( $v_s, v_p, []$ ));
6   return NonOverlapping(matches);

7 Function CompareNode( $v_s, v_p$ ):
8   return ( $d_{in}(v_s) = d_{in}(v_p) \vee d_{in}(v_p) = 0$ )  $\wedge$ 
           ( $d_{out}(v_s) = d_{out}(v_p) \vee d_{out}(v_p) = 0$ )  $\wedge$ 
           ( $c_{op}(v_s) = c_{op}(v_p)$ );

9 Function MatchNodes( $v_s, v_p, map$ ):
10  if CompareNode( $v_p, v_s$ ) then
11    map  $\leftarrow (v_p, v_s)$ ;
12    foreach  $\{p : p \in \text{parents}(v_p) \wedge p \notin map\}$  do
13      m  $\leftarrow []$ ;
14      foreach  $\{s : s \in \text{parents}(v_s) \wedge s \notin map\}$  do
15        | m.add(MatchNodes( $s, p, map$ ));
16      if  $m = []$  then return [];
17      else map.add( $m$ );
18    end
19    foreach  $\{p : p \in \text{childs}(v_p) \wedge p \notin map\}$  do
20      m  $\leftarrow []$ ;
21      foreach  $\{s : s \in \text{childs}(v_s) \wedge s \notin map\}$  do
22        | m.add(MatchNodes( $s, p, map$ ));
23      if  $m = []$  then return [];
24      else map.add( $m$ );
25    end
26    return map;
27 else return [] ;

```

5. Implementation

5.2.2. Passes

In order to enable offloading of the attention mechanism to ITA, a MHSA merge pass was introduced, utilizing the newly implemented graph matching mechanism. This pass merges all operations present in a MHSA operation into a monolithic node, which can be offloaded to the accelerator. Furthermore, to simplify the development of new kernels and identify implementation errors, a debug pass was added. This pass enables the printing of the intermediate values of a network and comparing them against a golden model. This feature was extensively utilized during the implementation of new kernels and proved to be immensely helpful.

By default, the re-quantization strategy used by Deeploy and QuantLab involves a `mul-add-div` approach, resulting in small quantization errors, as the addition operation is executed with higher precision. However, the CMSIS platform employs an `add-mul-div` approach, which means that networks trained using the CMSIS re-quantization strategy may not execute correctly on other platforms and vice versa due to differences in the rounding process. To address this platform compatibility issue, an optimization pass was introduced to transform networks trained with CMSIS re-quantization to run correctly on all platforms. However, it should be noted that the reverse transformation is not possible without losing information, which may potentially result in rounding errors.

5.2.3. Kernels

To enable Transformer networks to run on any platform, several additional kernels had to be implemented, including generic convolution, generalized matrix multiplication, and sum-reduction kernels. However, these generic kernels are intended to serve as a fall-back solution when no optimized platform-specific kernel is available. It should be noted that these fall-back kernels are currently not well-optimized, in terms of both memory access and computational efficiency.

Consequently, to boost the performance of the MemPool platform, both with and without ITA, parallelized convolution, maxpool, matrix multiplication, re-quantization, and softmax were implemented. However, currently, only the 8-bit matrix multiplication is highly optimized and makes use of the single instruction, multiple data (SIMD) instruction provided by the XPulping extension available on MemPool. A comparison of the baseline implementation and two improved versions are depicted in Figure 5.2. The baseline implementation is a parallelized version of the fallback version mentioned earlier, which consists of multiple nested for-loops and is not optimized. In contrast, the final version exploits data-level parallelism, features loop unrolling, and preloads data to L1 via DMA calls in case one of the matrices is located in L2. In an end-to-end deployment setting, the optimized matrix multiplication kernel achieves a peak performance of 832 MOp/s/core and on average 817 MOp/s/core for a $64 \times 64 \times 64$ matrix multiplication parallelized over 16 cores, and at a cluster frequency of 500 MHz. One operation is defined as one 8-bit multiplication or 32-bit addition. It should be noted that buffers allocated by Deeploy

5. Implementation

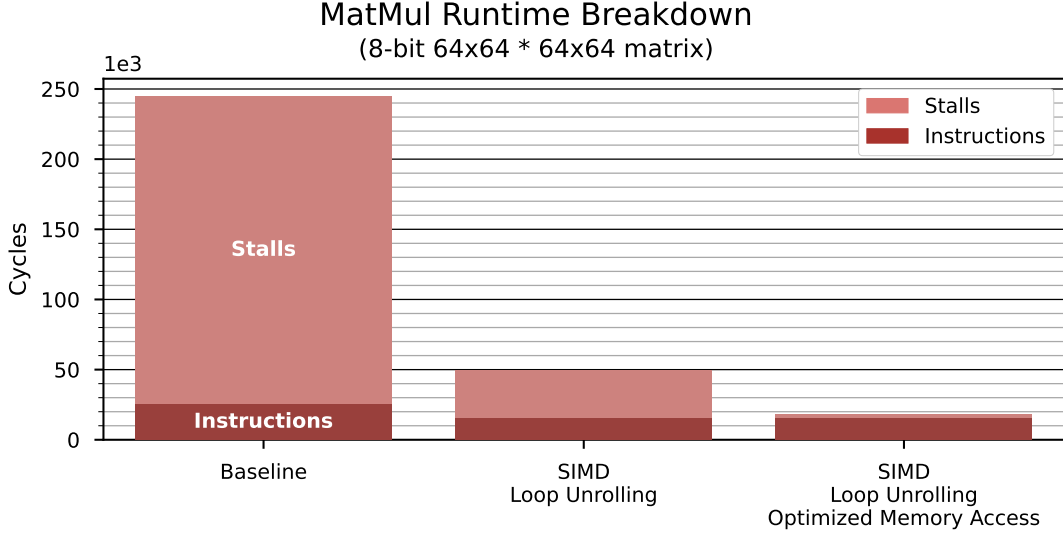


Figure 5.2.: Cycle breakdown of different matrix multiplication kernels running on MemPool. The $64 \times 64 \times 64$ matrix multiplication is parallelized over 64 cores in the baseline version and over 16 cores for the others. The baseline implementation uses nested for loops and does not exploit data-level parallelism, resulting in a peak performance of 95 MOp/s/core. The final version exploits data-level parallelism, loop unrolling, and optimized memory access and has a peak performance of 817 MOp/s/core.

are currently often suboptimally aligned, resulting in a degradation of performance due to increased L1 access time. However, if the memory access is optimally aligned, the peak performance can be increased at least up to 1.05 GOp/s/core [17]. Compared to the baseline, which achieves less than 95 MOp/s/core on peak, 22.8 MOp/s/core on average, and is parallelized over 64 cores, the optimized kernel provides a speed-up of at least $8\times$ while requires $4\times$ fewer cores.

Additionally, the performance of the attention operation can be further optimized by fusing re-quantization operations with the previous layers. Explicit re-quantization layers require many load and store instructions, leading to a significant increase in stall cycles. However, when fused with the previous layer, the re-quantization can be handled in the same kernel without significantly increasing the required memory bandwidth. To illustrate the benefits of fusing re-quantization operations, Figure 5.3 shows the cycle breakdown of the cluster during the inference of an attention layer parallelized over 16 cores, with and without fusing the re-quantization with the matrix multiplication. The left plot illustrates that the re-quantization operation involves numerous stall cycles. However, by fusing the re-quantization and matrix multiplication, the cycle count can be reduced by 28%, resulting in a $1.4\times$ speed-up.

Furthermore, to offload the attention layer to ITA, a new MHSA kernel was developed, which handles the transformation of inputs to the required memory format, configura-

5. Implementation

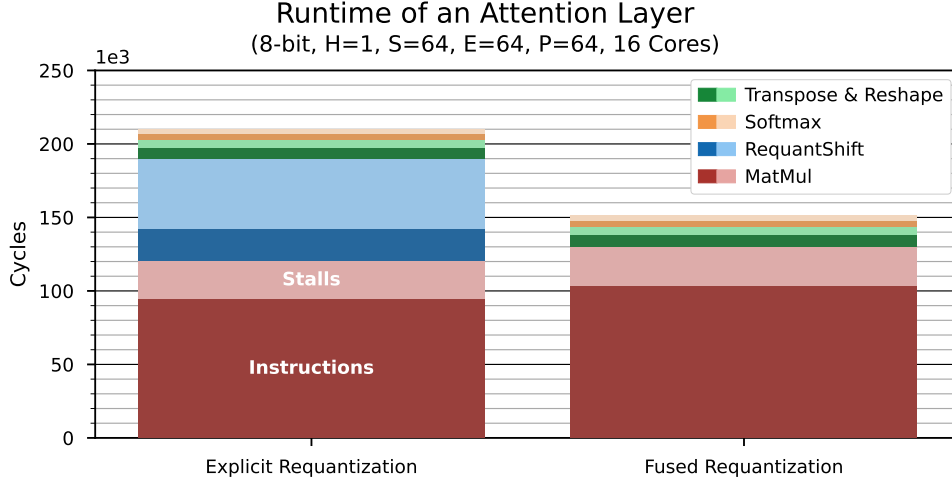


Figure 5.3.: Cycle Breakdown of an attention layer, showing the effect of fusing re-quantization into the previous kernel, leading to a speed-up of factor $1.4\times$.

tion of all parameters, and initiation of accelerator execution. After ITA completes the computation, the outputs are copied out of the accelerator memory and are available for head accumulation.

5.2.4. Limitations

The current implementation has several limitations that can be addressed to enhance the performance. The cycle breakdown shown in Figure 5.4 clearly indicates that the current implementation is causing numerous stall cycles, and some operations exhibit a very low arithmetic intensity, which significantly impacts the overall performance. Figure 5.5 provides a more detailed breakdown of the stall cycles.

As stated in [23], the attention layer can be calculated without any expensive explicit transpose operations. Therefore, it is possible to merge the matrix multiplications and transpositions in specialized kernels, improving the overall performance. An optimization pass can be added to Deeploy to transform the nodes and weights accordingly. Additionally, the fusion of re-quantization into the previous layer is currently only implemented for matrix multiplication, but it can easily be adapted to other layers such as convolution or non-linear activation functions.

Furthermore, there are several general optimization techniques that can be employed in the kernel and memory allocation, such as optimizing the buffer allocation strategy to minimize memory access time and further enhancing the already implemented kernel. Strategies like loop unrolling, using SIMD instructions, and increasing data reuse capabilities have already proved to be highly beneficial.

5. Implementation

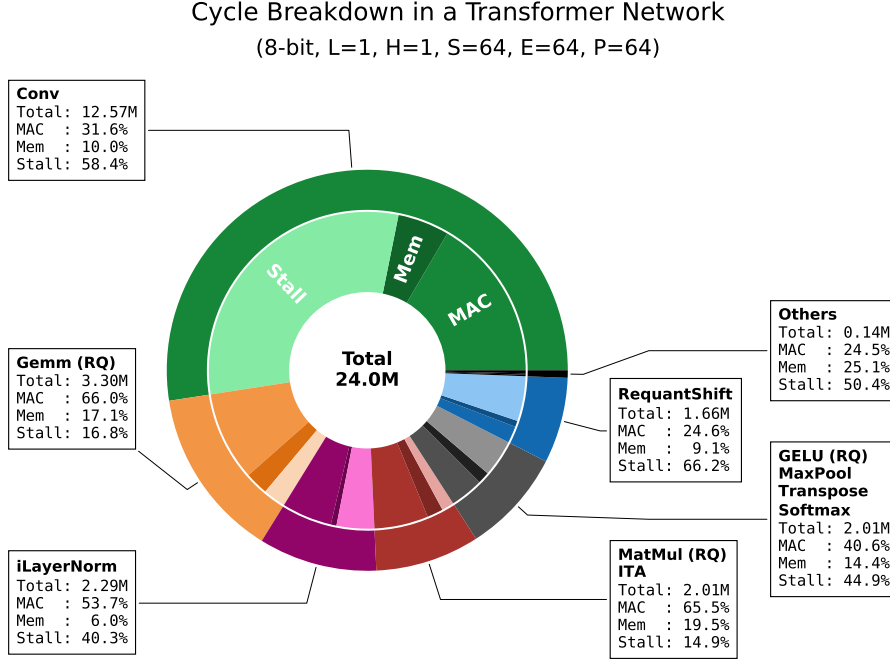


Figure 5.4.: Cycle breakdown of Transformer network, showing the contribution of arithmetic, memory, and stall cycles.

Additionally, all matrix dimensions for the MHSA with ITA are fixed at a size of 64. For smaller matrices, this constraint stems from the fact that padding with zeros will affect the result of the softmax function. To counter this, the matrix before the softmax layer must be padded with the most negative value, in this case, -128, to prevent any influence of the padding on the result. Unfortunately, the hardware does not yet support this feature. On the other hand, larger matrices pose another challenge that requires handling through matrix tiling. This technique involves dividing the larger matrix into smaller sub-matrices that can fit into the accelerator and performing computations on them sequentially.

Finally, the ITA cores are still coupled, which limits the support to only one head as mentioned in Section 5.3.1. Additionally, the software support for head scheduling and splitting, as well as the final head accumulation, is not yet fully integrated into Deeploy. This means that there is still a need to improve the overall software architecture to fully support multiple heads. Furthermore, ITA requires a specific memory layout for the input matrices, which results in additional overhead. Unfortunately, the standard DMA implementation in MemPool does not feature strided transfers, which further increases the overhead. However, it is possible to adapt the special DMA of ITA to directly load the inputs and transform them to the required format. This would reduce the overhead and improve the efficiency of the system.

5. Implementation

Stall Breakdown in a Transformer Network
(8-bit, L=1, H=1, S=64, E=64, P=64)

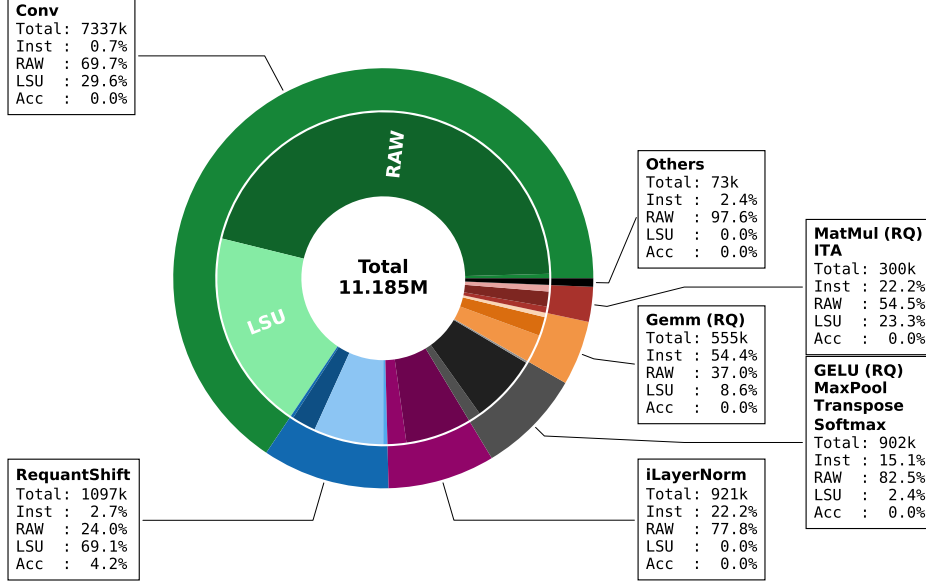


Figure 5.5.: Stall breakdown of a Transformer network, showing the contribution of instruction, read-after-write, load-store, and accelerator stall cycles of *Snitch*'s integer processing unit (IPU).

5.3. ITA Integration

A hardware implementation of ITA integrated into MemPool was already provided by [7]. However, additional modifications and features were added to the hardware to be able to use ITA with an end-to-end deployed Transformer network.

The softmax function¹ plays an important role in the attention mechanism present in Transformer networks. It normalizes the input matrix in a row-wise manner and transforms them into probabilities. It is defined as:

$$\text{softmax}(\mathbf{x})_i = \frac{e^{x_i - \max(\mathbf{x})}}{\sum_{j=1}^n e^{x_j - \max(\mathbf{x})}} \quad (5.1)$$

As explained in [7], for the quantized softmax function, there exists a maximum scaling factor ϵ for which some inputs will be mapped to zero. Consequently, the inputs can be clipped to values that result in a non-zero output, as depicted in Figure 5.6. This permits the use of a fixed $\epsilon_{max} = B/(2^B \log_2 e)$ during the softmax calculation with $\mathbf{x} = \epsilon_{max} \mathbf{x}_q$,

¹Useful visualization of the softmax function by Elliot Waite: <https://www.desmos.com/calculator/drqqhtb037>

5. Implementation

thus reducing the complexity of the hardware implementation. In addition, to enable symmetric uniform quantization, the inputs are shifted to center them around zero. However, this data-dependent clipping, shifting, and rescaling process either requires direct support from the hardware or can be estimated during training time and integrated into the re-quantization step before the softmax calculation.

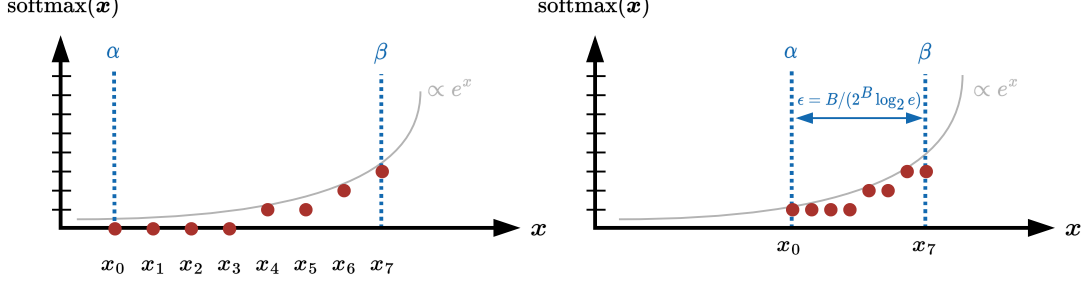


Figure 5.6.: Effect of quantizing the softmax activation function. (left) Some inputs will be mapped to zero if the input range is too large. (right) The inputs can be clipped to values that result in non-zero outputs.

Consequently, an additive bias term was added to ITA’s re-quantization module to facilitate the required shift operation and to align its functionality with the re-quantization behavior of QuantLab. Additionally, rounding capabilities were added to perform midpoints-away-from-zero rounding. Moreover, to enable dynamic configuration of the re-quantization parameters, additional memory-mapped configuration registers were added to MemPool on the system level, which allows configuring all four ITA-cores concurrently. Finally, to quantize networks with the novel streaming softmax implementation used in ITA, explained in Section 2.3.1, a new *ITAPartialMax* class was implemented in QuantLib and validated using the golden model provided by [7].

5.3.1. Limitations

This section highlights some limitations in the current implementation. Firstly, even though four ITA cores are available on MemPool, they are currently coupled and fetch the same parameter and inputs, thus computing the same operation. This implies that the computation of multiple heads is not parallelized, leading to suboptimal performance. Besides the obvious modification of the hardware, software support must also still be provided. This includes tiling, scheduling, synchronization, and accumulation of multiple heads by Deeploy. Unfortunately, there was not enough time to implement this feature, and thus Chapter 6 will analyze the result of an attention operation and estimate the potential performance of multiple heads.

Secondly, the re-quantization module currently employs a mul-div-add strategy, with all arguments in 8 bits and the operation performed in 32 bits. However, to ensure

5. Implementation

consistency with the default implementation for MemPool in Deeploy, the order of the operation should be adjusted to `mul-add-div` with a 32-bit bias parameter. This further reduces the error introduced by the re-quantization, as the addition is performed with higher precision.

Finally, the bias values for the linear layers are stored at 8-bit, even though, the result of the matrix multiplication is 24 bits, before being re-quantized back to 8-bit. Hence, the dynamic range of the bias is much lower than the output, effectively rendering them useless. As a solution, either 24-bit bias values can be used, or the 8-bit bias values can be quantized with a different scaling factor to match the dynamic range of the matrix multiplication result. This will ensure that the output values can be properly shifted and can be centered around zero during re-quantization.

5.4. Other Contributions

This section provides a summary of various contributions made to other projects during the course of the master’s thesis. Firstly, Banshee was modified to support the XPulping ISA extension and enhanced with a preliminary model of ITA, building upon the work of Samuel Riedel, Gamze Islamoglu, and Lorena Oswald. This modification enabled fast instruction-accurate simulations, reducing the average simulation time of a single-head attention layer from over 10 minutes to less than 20 seconds. This enhancement significantly expedited the development process. Secondly, the script used to generate visualizations of instruction traces was improved to handle larger files, enable compression, and support the new Perfetto trace viewer². The traces can be generated using either instruction-accurate simulators like Banshee or cycle-accurate simulators such as Siemens QuestSim[®] or Synopsys VCS[®].

In addition to the above-mentioned improvements, various modifications were made to QuantLab and its library QuantLib. The epsilon, sign, and *number of levels* propagation were extended to support additional layer types. Furthermore, a new exporter class was implemented, targeting Deeploy as the middleware and backend tool. During the exportation of the ONNX graph, the graph is properly shape-annotated and optimized using the ONNX-runtime optimizer configured to the basic level. These optimizations include constant folding, elimination of redundant nodes, and semantics-preserving node fusion³. Additionally, some code cleanup and bug fixes were implemented, and the frequency and content of logging messages were improved to provide a better user experience. Finally, support for PyTorch version 1.13.1 was extended and tested. Overall, these contributions made significant improvements to the efficiency, functionality, and user experience of the projects⁴.

²More information under <https://ui.perfetto.dev/>

³More information under [Graph Optimizations in ONNX Runtime](#)

⁴This version is available on the `devel` branch of the main repository on GitLab.

See <https://iis-git.ee.ethz.ch/spmatteo/quantlab/-/tree/devel>

Chapter 6

Results and Evaluation

This chapter presents the evaluation and results of the implementation explained in Chapter 5. Section 6.1 focuses on the performance of the attention operation on MemPool with and without ITA. The subsequent Section 6.2, evaluates the runtime of a full Transformer network based on the CCT architecture proposed by [24]. Finally, Section 6.3 compares our work with state-of-the-art Transformer accelerators and recent NVIDIA GPUs. Additionally, a qualitative comparison of Deeploy’s functionality to other deployment frameworks is provided.

6.1. End-to-End Multi-Head Self-Attention

To evaluate the performance of the MHSA operation on MemPool with and without ITA, a computational graph representing an attention layer was hand-crafted using the ONNX Python package. A golden model based on the work of [7] was utilized to generate the random inputs and calculate intermediate activation and output values. Table 6.1 presents the key parameters of the MHSA graph that are of particular importance.

Table 6.1.: Key parameters of the attention network.

Symbol	Parameter	Value
H	Number of attention heads	1
S	Sequence length	64
E	Embedding dimension	64
P	Projection dimension	64
B	Precision of inputs	8-bit
Bias	Projection layer bias	False

6. Results and Evaluation

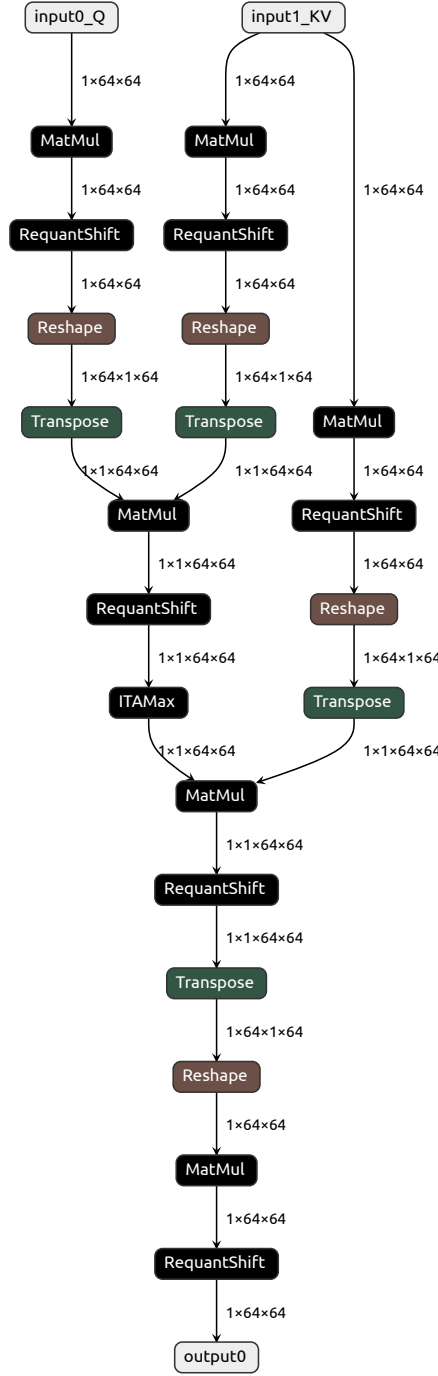


Figure 6.1.: ONNX representation of an 8-bit attention layer with linear input projection, attention mechanism, and linear output projection. The *ITAMax* layer represents the softmax implementation used in ITA without the partial streaming functionality.

6. Results and Evaluation

Due to the limitations mentioned in Chapter 5, only one head is used, and all dimensions are equal to 64. Additionally, because ITA only supports 8-bit biases on 24-bit values, it was decided not to use them. The network architecture is illustrated in Figure 6.1. The output of each matrix multiplication is re-quantized to 8-bit before being passed to the next layer. The *ITAMax* layer represents the softmax implementation present in ITA but running on the *Snitch* cores. Additionally, *ITAMax* omits the partial streaming functionality explained in Section 2.3.1 to simplify the implementation of the kernel in C code.

In the first experiment, the MHSA network shown in Figure 6.1 was deployed on Mem-Pool without ITA. During the deployment process, Deeploy optimizes the graph, resulting in the structure shown in Figure 6.2a. As mentioned in Chapter 5, the matrix multiplications are fused with the subsequent re-quantization layers to improve performance. In the second experiment, the same network was deployed with ITA, except that *ITAMax* is replaced with *ITAPartialMax* to distinguish between the version with and without the partial streaming functionality. During optimization, Deeploy transformed the original graph and folded all operations into a monolithic node that contained all the information necessary for computing the attention layer. Figure 6.2b illustrates the resulting network.

6.1.1. Performance

The performance evaluation of the deployed attention network was conducted through cycle-accurate simulations using Siemens QuestaSim[®]. The performance is defined as the ratio of active cycles spent to the number of operations. The active cycles exclude the cycles spent on waiting for wake-ups, memory de- and allocation of intermediate buffers, and cycles spent on reshape and transpose operations. The latter two are justified, as the memory de- and allocations cause an overhead of less than 3%, and [23] showed that no explicit data marshaling is required for calculating attention layers. Further, one operation was defined as either an 8-bit multiplication or a 32-bit addition, and the number of operations for the MHSA was calculated using:

$$\begin{aligned}
 n_{\text{op,MatMul}} &= H \cdot 2 \cdot N^3 \\
 n_{\text{op,RequantShift}} &= H \cdot 3 \cdot N^2 \\
 n_{\text{op,Softmax}} &= H \cdot 6 \cdot N^2 \\
 n_{\text{op,ReduceSum}} &= (H - 1) \cdot 2 \cdot N^2 \\
 n_{\text{op,MHSA}} &= 6 \cdot n_{\text{op,MatMul}} + 6 \cdot n_{\text{op,RequantShift}} \\
 &\quad + n_{\text{op,Softmax}} + n_{\text{op,ReduceSum}}
 \end{aligned} \tag{6.1}$$

with H the number of heads and all matrix dimensions equal to N . For $H = 1$ and $N = 64$, the total number of operation is $n_{\text{op,MHSA}} = 3244032$.

6. Results and Evaluation

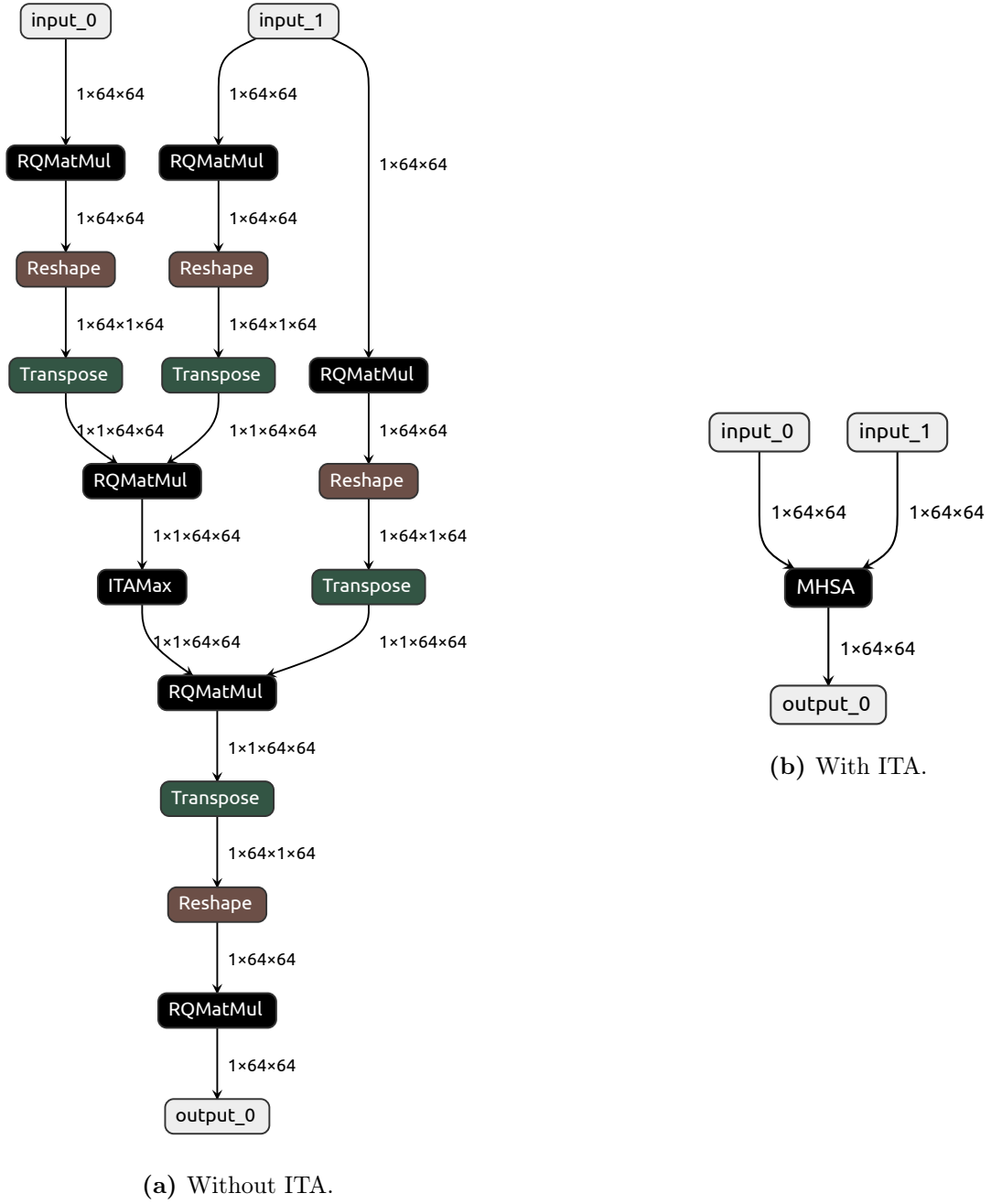


Figure 6.2.: Optimized and deployable graphs of the attention networks with and without ITA. Without ITA, the re-quantization and matrix multiplication are fused, whereas with ITA all operations are fused into a monolithic node which can be offloaded to the accelerator.

6. Results and Evaluation

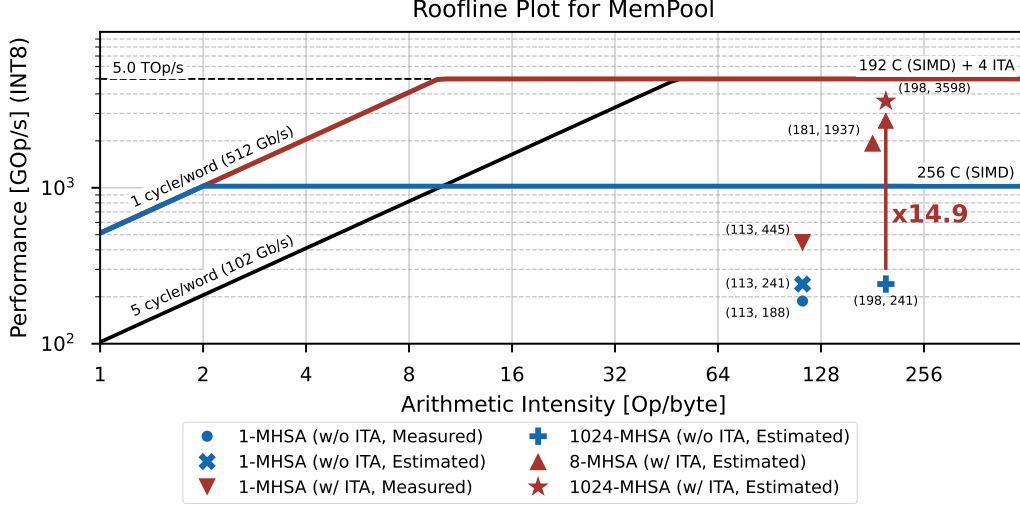


Figure 6.3.: Measured average core performance of the attention layer with one head, and the estimated performance when calculating multiple heads, both with and without ITA in an end-to-end deployment setting.

Figure 6.3 presents the performance and arithmetic intensity of the MHSA operation for one head as well as estimated values for eight and 1024 heads. The arithmetic intensity is the ratio of the number of operations to the amount of data that needs to be transferred in and out of the memory to perform these operations. In the roofline plot, the blue line represents the performance limit without ITA but with SIMD instructions, while the red line shows the performance limit with ITA. Each core can perform up to two 32-bit or eight 8-bit SIMD operations per cycle. Consequently, the performance limits for the 256 cores respectively 192 Snitch and 4 ITA cores configuration at a cluster frequency of 500 MHz are:

$$\begin{aligned}
 P_{\text{Limit}, 256\text{C}} &= 256 * 8 \text{ Op/Cycle} * 500 \text{ MHz} \\
 &= 1 \text{ TOp/s} \\
 P_{\text{Limit}, 192\text{C}+4\text{ITA}} &= (192 * 8 + 4 * 2112) \text{ Op/Cycle} * 500 \text{ MHz} \\
 &= 5.0 \text{ TOp/s}
 \end{aligned} \tag{6.2}$$

with one ITA core capable of executing the 3 244 032 Op in $6 * 256 = 1536$ cycles [7]. Additionally, the diagonal lines in the plot depict the memory bandwidth limit between the cores and the L1 scratchpad memory. The access time for each core ranges from one to five cycles per 32-bit value, depending on whether the local tile (1 cycle), local group (3 cycles), or remote group (5 cycles) is accessed without conflicts. Therefore, a kernel can either be memory-bound or compute-bound, depending on whether the arithmetic intensity is smaller or larger than the point where the memory bandwidth and performance limit intersect.

One-Head Self-Attention

The performance of the attention layer with one head (1-MHSA) and without ITA is 188 GOp/s. However, further optimization is possible, as discussed in Chapter 5. According to [17], it is possible to boost the performance of the matrix multiplication kernel at least up to 269 GOp/s, resulting in an overall performance of 241 GOp/s for the 1-MHSA layer. The performance with ITA, directly extracted from the simulation without any additional assumption, is only 23.5 GOp/s. However, assuming that the transformation of the input data for ITA can be handled by the ITA-DMA directly, and the outputs from ITA can be transferred via DMA calls, a performance of 445 GOp/s can be achieved. Therefore, by using ITA, the performance of MemPool improves $1.8\times$ times compared to the optimized core-only version when processing a single attention head. Additionally, when taking into account all operations, including the transpose operations and the memory transfers from and to ITA, paralleling them over 16 cores, the runtime can be improved up to 19 times with ITA.

Multi-Head Self-Attention

The plot also shows the estimated performance when calculating multiple heads using all four ITA-cores. The estimation assumes that the *Snitch* cores must compute each head sequentially and the performance of the matrix multiplication kernel is 269 GOp/s instead of 209 GOp/s based on [17]. Further, it assumes that all data to and from ITA is directly transformed by a DMA, and multiple iterations performed by one ITA-core can be pipelined as explained in [7]. Consequently, the performance without ITA for the attention layer with more heads is the same as for the 1-MHSA version. However, when using ITA, an estimated average performance of 1937 GOp/s with eight heads (8-MHSA) can be achieved, resulting in $8\times$ higher performance. When further increasing the number of heads to 1024, an asymptotic performance increase of up to $15\times$ to 3598 GOp/s can be achieved.

6.1.2. Efficiency

By extrapolating the results from the simulation to multiple heads the performance and efficiencies listed in Table 6.2 can be achieved. The power requirements for the cores are based on [17] due to the similar workload and linearly scaled to the number of active cores. Furthermore, the area of MemPool is 11.78 mm^2 [16] and the number for ITA are based on [7]. The decrease in energy and area efficiency is caused by the additional cores required to perform the head accumulation. However, if more than eight heads are calculated, the additional power and area required by the cores to perform the head accumulation are compensated by the increase in performance. This is because the ITA-cores interleave the loading of the weights with the previous computation cycle.

6. Results and Evaluation

Table 6.2.: Comparison of the estimated performance and efficiency of the attention operation deployed with Deeploy.

Configuration	Attention Heads	Performance [TOp/s]	Energy Efficiency [TOp/s/W]	Area Efficiency [TOp/s/mm ²]
256 C	H	0.241 ^a	0.173 ^a	0.020 ^a
1 C + 1 ITA	1 ^b	0.445	3.516	0.980
16 C + 4 ITA	4	1.322	2.316	0.558
16 C + 4 ITA	8	1.937	3.393	0.818
16 C + 4 ITA	64	3.267	5.722	1.380
16 C + 4 ITA	1024	3.598	6.302	1.520
Speedup	1024	14.9×	36.4×	74.4×

^a Neglecting the overhead of the head accumulation.

^b No head accumulation is required.

In summary, by increasing the number of heads up to 1024, a performance increase for the attention operation of up to 14.9× can be achieved. Due to the energy- and area-efficient design of ITA, the energy and area efficiency can be increased by up to a factor of 36.6× and 74.4×, respectively.

6.2. End-to-End Transformer Network

After the first few weeks of the project, the original plan to deploy an I-BERT model [2] on MemPool was replaced with a more suitable option, a CCT architecture as proposed by [24]. The decision was made because ITA currently only supports sequence, embedding, and projection dimensions equal to 64, while I-BERT operates on much larger dimensions. Therefore, an alternative architecture was chosen that would not require considerable modifications to the accelerator. To evaluate the end-to-end deployment of Transformer networks on MemPool, a test network based on the CCT architecture was trained on the CIFAR10 dataset and quantized using QuantLab. The structure of the FP network is shown in Figure 6.4. Additionally, the ONNX graph of the quantized network is provided in Figure E.1 in Appendix E.

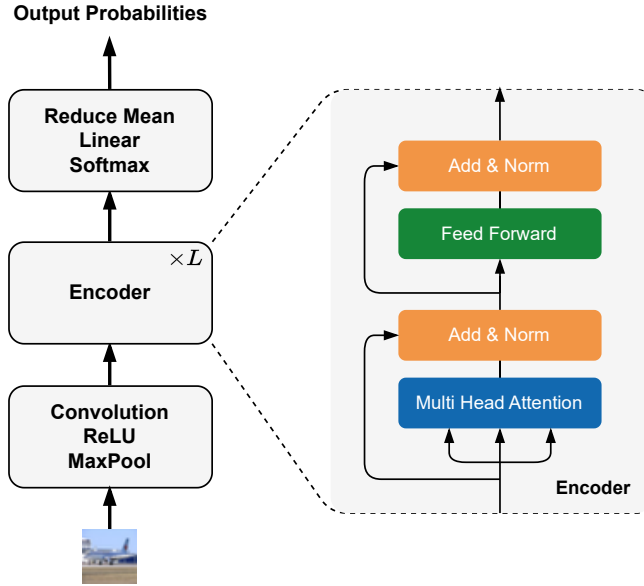


Figure 6.4.: Architecture of the ICCT network, based on the topology proposed by [24]. The ONNX graph of the quantized network is provided in Appendix E.

The network, further referred to as ICCT, was implemented as a new topology in QuantLab, allowing easy customization through keyword arguments in a JSON configuration file. The key parameters of the evaluation network are listed in Table 6.3. The quantization process utilized the TQT algorithm, resulting in an accuracy loss of 4.2 p.p. compared to the FP network achieving 71.7% accuracy on CIFAR10. It's important to note that vision transformers often perform badly on small-scale datasets without being pre-trained on larger images and then fine-tuned on the small ones [25]. However, the focus was not specifically put on optimizing the accuracy and quantization error, but on testing the entire deployment workflow.

6. Results and Evaluation

Table 6.3.: Key parameters of ICCT.

Parameter	Symbol	Value
H	Number of attention heads	1
S	Sequence length	64
E	Embedding dimension	64
P	Projection dimension	64
L	Number of attention layers	1
Bias	Projection layers bias in attention operation	False

6.2.1. Runtime

Figure 6.5 shows the runtime of the quantized network running on 16 cores with and without offloading the attention operation to ITA. The inclusion of ITA results in a 10 % reduction in runtime. However, this speedup is relatively modest due to two main factors: Firstly, the attention operation accounts for only 35 % of the total required operations. Secondly, the convolution kernel is not optimized, leading to significant stalls and contributing to 52 %, respectively 58 % of the overall runtime. Consequently, by optimizing the kernels further and utilizing networks with multiple attention heads, more of the potential performance boost offered by ITA can be unlocked.

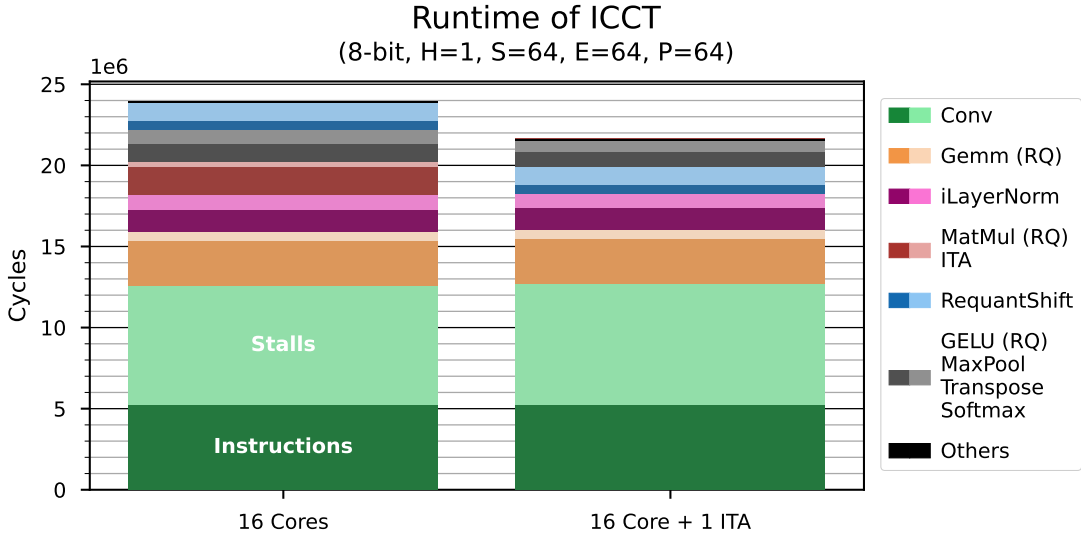


Figure 6.5.: Runtime of ICCT with and without ITA. With ITA, the runtime can be decreased by 10 % with ITA. Some kernels (RQ) are fused with the subsequent re-quantization layer, however, not all are fully optimized.

6.3. Comparison with State of the Art

This section compares our work with state-of-the-art Transformer accelerators and modern GPUs for large-scale machine learning applications. The standalone version of ITA proposed by Gamze et al. [7] integrates one ITA core into a system with 64 KiB SRAM, allowing for double-buffering of all matrices. The integrated version examines MemPool with four ITA. The performance metrics consider either all 192 or only 16 *Snitch* cores, as currently, only 16 cores are necessary for the head accumulation of ITA’s outputs.

6.3.1. Attention Operation

A comparison of the attention operation running on MemPool with ITA to state-of-the-art Transformer accelerators is presented in Table 6.4. MemPool with ITA outperforms all other accelerators in terms of throughput performance. Additionally, depending on the configuration, our work achieves an energy efficiency of up to 7.49 TOP/s/W. It is important to note that except for Keller et al. [26], all other approaches assume sparsity. In addition, [27] and [28] use compute-in-memory (CIM) architectures to improve efficiency.

Table 6.4.: Comparison of MemPool with ITA to state-of-the-art Transformer accelerators.

	Wang et al. [29]	Keller et al. [26]	Tu et al. [27]	Liu et al. [28]	This work	
					Standalone [7]	Integrated
Type	Digital MAC	Digital MAC	Digital CIM	Digital CIM	Digital MAC	Digital MAC
Sparsity	✓	✗	✓	✓	✗	✗
Technology [nm]	28	5	28	28	22	22
Area [mm ²]	6.82	0.153	14.36	3.93	0.407	2.37 ⁷ -11.8 ⁸
Voltage [V]	0.56-1.1	0.46-1.05	0.6-1.0	0.64-1.03	0.8	0.8
Frequency [MHz]	50-510	152-1760	85-275	20-230	500	500
Data formats	INT12	INT8	INT8/16	INT8	INT8	INT8
On-chip memory [kB]	336	141	192	96	67.8	320 ⁷ -1024 ⁸
Power [mW]	12.06-272.8	-	29.83-152.75	8.27-250.65	121	571 ⁷ -1527 ⁸
Throughput [TOP/s]	0.52-4.07 ¹	1.8 ⁴	3.55 ^{3,4}	3.33 ⁶	1.02	3.62⁷-4.98⁸
Energy efficiency [TOP/s/W]	1.91-27.56 ²	39.1 ⁵	48.4-101.1^{3,5}	1.96-25.22 ^{5,6}	8.46	3.26 ⁸ -7.49 ⁷
Area efficiency [TOP/s/mm ²]	0.0765-0.597 ²	11.7⁴	0.06-0.25 ^{3,4}	0.85 ^{4,6}	2.52	0.48 ⁸ -1.80 ⁷
Area efficiency ^a [TOP/s/MGE]	0.0247-0.192 ²	0.121 ⁴	0.034-0.165 ^{3,4}	0.562^{4,6}	0.500	0.095 ⁸ -0.361 ⁷

^a Gate-equivalents (GE) of other technologies are scaled based on the GE of the 22 nm technology.

¹ Measured at 1.1 V, 510 MHz. The highest throughput is measured for $A \times V$ assuming 90% near-zero and zero probabilities.

² The lowest value measured at 1.1 V, 510 MHz for linear layers that compute Q, K, V and the highest value measured at 0.56 V, 50 MHz for $A \times V$ assuming 90% near-zero and zero probabilities.

³ Level of sparsity not stated.

⁴ At highest supply voltage.

⁵ At lowest supply voltage.

⁶ Assuming 75% sparsity in weight and 92% sparsity in attention span and excluding off-chip memory access.

⁷ System with 16 *Snitch* cores and 4 ITA cores.

⁸ System with 192 *Snitch* cores and 4 ITA cores, assuming 100% utilization.

When comparing the accelerators at the same voltage using V_{dd}^2 scaling, MemPool with ITA is only $1.7\times$ less efficient than [26]. In terms of area efficiency in gate-equivalent

6. Results and Evaluation

(GE), the integrated ITA version surpasses all other accelerators except for Liu et al. [28], which utilizes significantly less on-chip memory.

Furthermore, a comparison to state-of-the-art GPUs specifically the NVIDIA A100 [30] and the latest NVIDIA H100 [31], targeting large-scale machine learning infrastructure, is provided in Table 6.5. MemPool with ITA achieves similar INT8 energy efficiency despite being implemented in much less advanced technology. Remarkably, in terms of area efficiency in GE, our work is 39–46 times more efficient than the NVIDIA A100 and 15–17 times better than the NVIDIA H100 GPU.

Table 6.5.: Comparison of MemPool with ITA to state-of-the-art GPUs.

	NVIDIA A100 [30]	NVIDIA H100 [31]	This work	
			Standalone [7]	Integrated ³
Technology [nm]	7	5 ¹	22	22
Release	2020	2022	2022	2022
Area [mm ²]	826	814	0.407	2.37
Throughput [TOp/s]	624	1600	1.02	3.62-4.28
Energy efficiency ^a [TOp/s/W]	1.56-3.12 ²	4.57-9.14 ²	8.46	6.34-7.49
Area efficiency [TOp/s/mm ²]	0.767	1.966	2.52	1.53-1.80
Area efficiency ^b [TOp/s/MGE]	0.0078	0.0203	0.500	0.306-0.361

^a INT8 tensor computation without sparsity.

^b Gate-equivalents (GE) of other technologies are scaled based on the GE of the 22 nm technology.

¹ TSMC 5 nm process customized for NVIDIA (4N)

² Range is based on Špet'Ko et al. [32] and information provided in the datasheet using the Thermal Design Power (TDP).

³ System with 16 *Snitch* cores and 4 ITA cores.

⁴ Depending on the number of attention heads.

6.3.2. Deploy

To the best knowledge of the author, there is a lack of comparative studies analyzing the functionality of deployment frameworks for embedded systems. In order to address this gap, a qualitative comparison of the capabilities of Deeploy to the frameworks presented in Chapter 3 is provided in Table 6.6.

Although TFLM is widely used and benefits from its publicly available code base, it lacks support for multicore architectures with multiple memory levels or heterogeneous systems with accelerators. While it is possible to implement custom operators in TFLM, it can be challenging and often requires extensive modifications. On the other hand, STM32Cube.AI offers broad support for various input formats and is considered user-friendly. However, it is limited to STM32 microcontrollers, its code generation tool is not open source, and it lacks many advanced features. Both GWT GAPflow and DORY support heterogeneous multicore systems with multiple user-managed memory levels and facilitate matrix tiling. However, part of GWT GAPflow is proprietary, and both tools require a deep understanding to incorporate custom operators. Additionally, DORY

6. Results and Evaluation

only supports up to three concurrent branches in the network, considerably restricting its suitability for advanced research applications.

Deeploy offers extensive support for a wide range of platforms and is specifically designed to be modular and extensible. This design philosophy enables easy integration of new platforms, the addition of custom operators, and the development of extensions to incorporate new functionality. The framework demonstrates its capability to handle heterogeneous multicore systems with multiple user-managed memory hierarchies, as exemplified in this work. Moreover, ongoing efforts are dedicated to the development of new extensions, such as tiling, which will further enhance the framework’s capabilities. Deeploy also integrates support for mixed precision and mixed type inference. Additionally, it is worth noting that an open-source release of Deeploy is planned, ensuring accessibility and fostering collaborative development in the future.

Table 6.6.: Comparison of Deeploy with State-of-the-Art Deployment Frameworks

Work	Platforms	Input	Output	Open-Source	Multi-Level Multi-Core	Custom Operator	Heterogeneous System Matrix Tiling
TFLM [33]	ARM Cortex-M	TensorFlow Lite	C++ 11	✓	✗	✗	≈ ^a ✗ ✗
STM32Cube.AI	STM32	PyTorch, Caffe, TensorFlow (Lite), ConvNet.js, Lasagne, ONNX	C99	✗	✗	✗	✗ ✗ ✗
GWT GAPflow	GAP	TensorFlow Lite	C	✗	✓	✓	≈ ^a ✓ ≈ ^b
DORY [20]	GAP, PULP	ONNX	C90	✓	✓	✓	≈ ^a ✓ ≈ ^b
Deeploy	PULP, MemPool, ARM Cortex-M	ONNX	C99	≈ ^c	✓	✓	✓ ✓ ✓

^aAdding custom operators can be quite challenging, ^bNo generic accelerator support,

^c Open-source release planned

Conclusion and Future Work

In this work, we present an end-to-end deployment workflow facilitating seamless deployment of neural networks on heterogeneous many-core systems with multiple user-managed memory hierarchies. To demonstrate the capabilities of the toolchain, we integrate MemPool with ITA in our workflow with QuantLab and Deeploy and deploy a Transformer network based on the CCT architecture. The introduction of new features in Deeploy, such as isomorphic graph matching, optimization passes, and kernels, enables automatic offloading of the MHSA operation in a Transformer network to ITA, resulting in significant performance improvements. The utilization of ITA in MemPool unlocks up to $15\times$ more performance, achieves $36\times$ higher energy efficiency and is up to $74\times$ more area efficient. Furthermore, the extensions and modifications made in Deeploy, ITA, QuantLab, and Banshee unlock a lot of potential for future optimizations.

In summary, the following contributions were made:

- We provide a pipeline to deploy neural networks on heterogeneous many-core systems. This includes quantization, automatic hardware-aware graph optimization, and optimized C code generation.
- We demonstrate the end-to-end deployment of a compact convolutional transformers (CCT) network on MemPool with the Integer Transformer Accelerator (ITA).
- We evaluate the performance of the automatically deployed attention operation on MemPool with and without ITA. We show that MemPool with ITA achieves up to $15\times$ higher throughput and $36\times$ higher energy efficiency compared to the original version of MemPool.
- We compare our work with state-of-the-art transformer accelerators and recent NVIDIA GPUs and show that ITA is a competitive alternative for non-spares multi-head attention calculation and that we achieve similar energy efficiency than modern GPUs while maintaining a much higher area efficiency.

7. Conclusion and Future Work

7.1. Future Work

To continue advancing our research and development, there are several potential avenues that can be explored: One area of improvement is the optimization of kernels. This involves avoiding transpositions, improving data reuse, utilizing SIMD instructions whenever possible, and applying loop unrolling techniques. These optimizations can significantly enhance the performance and efficiency of the kernels, as shown in this work.

Additionally, we suggest several enhancements to ITA and its integration to Deeploy. Firstly, decoupling the ITA cores and enabling parallel computation of multiple heads is not yet fully implemented. This also includes scheduling, synchronization, and the accumulation of the results on the Snitch cores. Secondly, to enhance the softmax module, we propose incorporating dynamic re-quantization to fix the input scaling factor before the softmax calculation. This adaptive rescaling mechanism would allow us to dynamically shift and rescale the inputs, and hopefully, further improve the accuracy and performance of the softmax module in ITA. Further, we can improve the throughput by splitting and reshaping ITA’s inputs using a DMA engine. Lastly, switching the biases for the linear layer in ITA to 24-bit biases would ensure that the output values can be properly shifted and can be centered around zero during re-quantization.

Another important direction for future work is to support matrix tiling and MHSA calculation of larger matrices utilizing the streaming partial softmax module present in ITA. This will require additional modifications to the hardware and Deeploy. Furthermore, the model of ITA in Banshee can be improved by supporting multiple ITA cores and improving the timing model.

Appendix	A
----------	----------

Task Description



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Institut für Integrierte Systeme
Integrated Systems Laboratory

MASTER THESIS AT THE DEPARTMENT OF
INFORMATION TECHNOLOGY AND ELECTRICAL ENGINEERING
FALL SEMESTER 2022

Transformer Deployment on Heterogeneous Many-Core Systems

November 11, 2022

Victor Jung (IIS), ETZ J 69.2, jungvi@iis.ee.ethz.ch
Gamze İslamoğlu (IIS), ETZ J 68.2, gislamoglu@student.ethz.ch

Handout: 21.11.2022
Due: 22.05.2023

The final report will be submitted in electronic format. All copies remain property of the Integrated Systems Laboratory.

1 Introduction

The demand for high performance under a limited power budget has led to the development of heterogeneous platforms like Apple M2 [1] and Qualcomm Snapdragon [2] combining multi-core processors and accelerators. While general-purpose processors execute sequential code, highly parallel specific tasks such as encryption, signal processing, and machine learning are offloaded to accelerators, resulting in improved performance and energy efficiency.

Transformer is a relatively new deep learning model proposed in 2017 and provides higher accuracy compared to recurrent neural networks (RNNs) and convolutional neural networks (CNNs) in natural language processing (NLP) and computer vision tasks [3, 4, 5]. The self-attention mechanism is a key component of the model but results in a high computational load and a complicated operational scheme. For this reason, we developed an Integer Transformer Accelerator (ITA) that can efficiently perform self-attention and integrated it into a many-core system called MemPool.

MemPool [6] is an image signal processor (ISP) designed at ETH. Originally, it boasts 256 lightweight 32-bit Snitch cores developed at ETH [7]. They implement the RISC-V instruction set architecture (ISA), which is a modular and open ISA [8]. MemPool with ITA includes 192 cores and 4 ITA cores that accelerate the execution of attention for transformer workloads.

The goal of this thesis is to deploy transformer models to MemPool with ITA using DumpO.

2 Project Goals

The main tasks of this project are:

- **T1: Deployment of a Dense layer to MemPool** In the first task, we aim to generate a basic deployment pipeline and test it with a dense layer. The student will add MemPool as a new target platform for DumpO, obtain the code for the execution of the dense layer, and simulate it in MemPool.
- **T2: Quantization of Transformer model** In the second task, we select and quantize an I-BERT Encoder for MemPool with ITA. The student will use QuantLab¹ for quantization and prepare the model for deployment.
- **T3: Deployment of I-BERT Encoder on ITA with MemPool** The student will then extend the deployment pipeline made in T1 to generate the accelerator code to control ITA. Additionally, he/she will have to parallelize and tile the execution according to ITA's constraints.
- **T4: Deployment of I-BERT Encoder on MemPool only** The student will extend the platform to run the I-BERT encoder using only Snitch cores. We will provide the optimized kernels, the student will have to decide on a parallelization and tiling strategy for the execution of the MHSA.
- **T5: Measuring and benchmarking** The student will evaluate the performance of the model on MemPool and ITA.

¹<https://github.com/pulp-platform/quantlab>

- **T6 (Additional): Deployment of End-to-End Transformers to MemPool with ITA** The student will extend again the pipeline to support full deployment of an End-to-End Transformer and will evaluate the performances

3 Deliverables

1. **D1: Quantized Transformer Model**
2. **D2: Extended code for DumpO targeting MemPool with ITA.**
3. **D3: Report and presentation detailing the key performance metrics and comparison to state-of-the-art implementations.**

4 Project Realization

4.1 Project Plan

Within the first month of the project, you will be asked to prepare a project plan. This plan should identify the tasks to be performed during the project and set deadlines for those tasks. The prepared plan will be a topic of discussion in the first week's meeting between you and your advisers. Note that the project plan should be updated constantly depending on the project's status.

4.2 Meetings

Weekly meetings will be held between the student and the assistants. The exact time and location of these meetings will be determined within the first week of the project in order to fit the student's and the assistants' schedules. These meetings will be used to evaluate the status and progress of the project. Besides these regular meetings, additional meetings can be organized to address urgent issues as well.

4.3 Report

Documentation is an important and often overlooked aspect of engineering. One final report has to be completed within this project. The common language of engineering is de facto English. Therefore, the final report of the work is preferred to be written in English. Any form of word processing software is allowed for writing the reports, nevertheless, the use of $\text{\LaTeX}$ with Tgif² or any other vector drawing software (for block diagrams) is strongly encouraged by the IIS staff.

Final Report The final report has to be presented at the end of the project and a digital copy needs to be handed in. Note that this task description is part of your report and has to be attached to your final report.

²See: <http://bourbon.usc.edu:8001/tgif/index.html> and <http://www.dz.ee.ethz.ch/en/information/how-to/drawing-schematics.html>.

4.4 Presentation

There will be a presentation (20 min presentation and 5 min Q&A) at the end of this project in order to present your results to a wider audience. The exact date will be determined towards the end of the work.

References

- [1] Apple. (2022, Jun.) Apple unveils M2, taking the breakthrough performance and capabilities of M1 even further. [Online]. Available: <https://www.apple.com/newsroom/2022/06/apple-unveils-m2-with-breakthrough-performance-and-capabilities/>
- [2] Qualcomm. Snapdragon platforms. [Online]. Available: <https://www.qualcomm.com/snapdragon>
- [3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2017. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” 2020. [Online]. Available: <https://arxiv.org/abs/2010.11929>
- [5] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” 2020. [Online]. Available: <https://arxiv.org/abs/2012.12877>
- [6] M. Cavalcante, S. Riedel, A. Pullini, and L. Benini, “MemPool: A shared-L1 memory many-core cluster with a low-latency interconnect,” in *2021 Design, Automation, and Test in Europe Conference and Exhibition (DATE)*, 2021, pp. 701–706.
- [7] F. Zaruba, F. Schuiki, T. Hoefer, and L. Benini, “Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads,” *IEEE Transactions on Computers*, vol. 70, no. 11, pp. 1845–1860, 2021.
- [8] A. Waterman and K. Asanovic, *The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 20191213*, RISC-V Foundation, Dec. 2019.

Zürich, November 11, 2022

Prof. Dr. Luca Benini

Appendix	B
----------	----------

Declaration of Originality



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Transformer Deployment on Heterogeneous Many-Core Systems

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):

Wiese

First name(s):

Philip

With my signature I confirm that

- I have committed none of the forms of plagiarism described in the '[Citation etiquette](#)' information sheet.
- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date

Zurich, 22.05.2023

Signature(s)

P. Wiese

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.

Appendix C

Source Code

C.1. Graph Matching Python Code

Listing C.1: Compare Nodes

```
1 # Check if nodes are identical
2 def attributes_are_equal(pn: gs.Node, gn: gs.Node) -> bool:
3     ret = True
4     pn_inputs = [pn_input for pn_input in pn.inputs if len(pn_input.inputs) > 0]
5     gn_inputs = [gn_input for gn_input in gn.inputs if len(gn_input.inputs) > 0]
6     if len(pn.inputs) != 1 or len(pn.inputs[0].inputs) != 0:
7         ret &= (len(pn_inputs) == len(gn_inputs))
8
9     pn_outputs = [pn_output for pn_output in pn.outputs if len(pn_output.outputs)
10                    > 0]
11     gn_outputs = [gn_output for gn_output in gn.outputs if len(gn_output.outputs)
12                    > 0]
13     if len(pn.outputs) != 1 or len(pn.outputs[0].outputs) != 0:
14         ret &= (len(pn_outputs) == len(gn_outputs))
15
16     ret &= (pn.op == gn.op)
17     return ret, (pn_inputs, gn_inputs, pn_outputs, gn_outputs)
```

Listing C.2: Match Nodes

```
1 def _match_nodes(self,
2                   ctxt: NetworkContext,
3                   pn: gs.Node,
4                   gn: gs.Node,
5                   nodes_map: dict = None,
6                   direction: Literal["Forward", "Reverse"] = 'Forward'):
7     assert direction in ["Forward", "Reverse"], f'"{direction}" is not a valid_
8         matching_direction!'
9
10    nodes_map = {} if nodes_map is None else nodes_map
11
12    ret, (pn_inputs, gn_inputs, pn_outputs, gn_outputs) = attributes_are_equal(pn,
13                                                                                  gn)
```

C. Source Code

```

12     if not ret:
13         return None
14
15     # Add candidate match to list
16     nodes_map[pn.name] = gn
17
18     # Check if we are done
19     if direction == 'Reverse':
20         # If our pattern is fully matched until here
21         if len(pn_inputs) == 0:
22             return nodes_map
23         # If our pattern is not fully matched completely but we reached leaf node
24         if len(pn_inputs) > 0 and len(gn_inputs) == 0:
25             return None
26
27     if direction == 'Forward':
28         # If our pattern is fully matched until here
29         if len(pn_outputs) == 0:
30             return nodes_map
31         # If our pattern is not fully matched completely but we reached leaf node
32         if len(pn_outputs) > 0 and len(gn_outputs) == 0:
33             return None
34
35     # If pn and gn have multiple parent nodes, we have want to traverse upwards
36     if len(pn.inputs) > 0 and len(gn.inputs) > 0:
37         for pn_input in pn.inputs:
38             # Check if parent node of pn is constant or input node (in this case
39             # it has no additional inputs)
40             # and if node was already matched
41             if len(pn_input.inputs) > 0 and pn_input.inputs[0].name not in
42                 nodes_map.keys():
43                 tmp = None
44             for gn_input in gn.inputs:
45                 # Check if parent node of gn is constant or input node (in
46                 # this case it has no additional inputs)
47                 # and if node was already matched
48                 if len(gn_input.inputs) > 0 and gn_input.inputs[0] not in
49                     nodes_map.values():
50                     # Search for valid subgraphs
51                     tmp = self._match_nodes(ctxt,
52                                             pn_input.inputs[0],
53                                             gn_input.inputs[0],
54                                             nodes_map,
55                                             direction = 'Reverse')
56             if tmp is not None:
57                 nodes_map = tmp
58
59         # If it was not possible to map parent node of pn to a parent node
60         # of gn
61         if tmp == None:
62             return None
63
64     # If it was possible to map all parent nodes of pn to a parent node of gn
65     if direction == 'Reverse':
66         return nodes_map
67
68     # If pn and gn have multiple child nodes, we have want to traverse downwards
69     if len(pn.outputs) > 0 and len(gn.outputs) > 0:
70         for pn_output in pn.outputs:
71             # Check if parent node of pn is is output node (in this case it has no
72             # additional outputs)
73             # and if node was already matched

```

C. Source Code

```

68         if len(pn_input.outputs) > 0 and pn_input.outputs[0].name not in
69             nodes_map.keys():
70             tmp = None
71             for gn_input in gn.outputs:
72                 # Check if parent node of gn is is output node (in this case
73                 # it has no additional outputs)
74                 # and if node was already matched
75                 if len(gn_input.outputs) > 0 and gn_input.outputs[0] not in
76                     nodes_map.values():
77                     # Search for valid subgraphs
78                     tmp = self._match_nodes(ctxt,
79                                             pn_input.outputs[0],
80                                             gn_input.outputs[0],
81                                             nodes_map,
82                                             direction = 'Forward')
83
84                 if tmp is not None:
85                     nodes_map = tmp
86
87             # If it was not possible to map parent node of pn to a parent node
88             # of gn
89             if tmp == None:
90                 return None
91
92             # If it was possible to map all child nodea of pn to a child node of gn
93             if direction == 'Forward':
94                 return nodes_map
95
96     assert False, "This_statement_should_never_be_reached!"

```

Listing C.3: Match Graph

```

1 def match_graph(self, ctxt: NetworkContext, graph: gs.Graph):
2     # this function returns a list of non-overlapping matches of self.p
3     # in gm, which is first traced with self.trace. Any matches which
4     # overlap previous matches are discarded.
5     self.searched_graph = graph
6     all_matches = []
7     matched_nodes = set()
8
9     def match_overlaps_with_previous(match):
10         return any(n.name in matched_nodes for k, n in match.nodes_map.items())
11
12     for node in self.searched_graph.nodes:
13         match = self.matches_subgraph_from_anchor(ctxt, node)
14         if match is not None:
15             if not match_overlaps_with_previous(match):
16                 all_matches.append(match)
17                 for k, n in match.nodes_map.items():
18                     matched_nodes.add(n.name)
19     return all_matches
20
21 def matches_subgraph_from_anchor(self, ctxt: NetworkContext, anchor: gs.Node):
22     # similar to the fx method, except the nodes_map is not a member of the
23     # class
24     match = self._match_nodes(ctxt, self.pattern_anchor, anchor)
25
26     if match is not None:
27         if len(match.keys()) != len(self.p.nodes):
28             mm = None
29         else:
30             mm = Match(anchor = anchor, nodes_map = match)
31     else:

```


C. Source Code

```
32     mm = None
33     return mm
```

Appendix D

MemPool Memory Layout

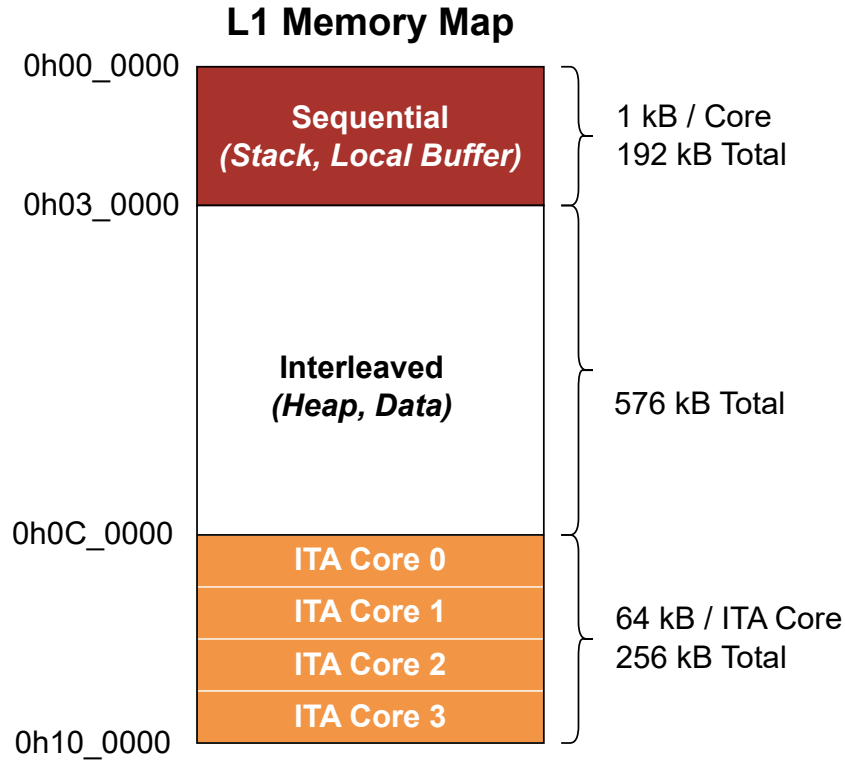


Figure D.1.: MemPool memory layout with ITA in the default configuration, assuming 192 compute cores, 4 ITA cores, and 1024B sequential memory per core. Each ITA-cores replaced four of the original *tiles*. The addressing scheme of the sequential and interleaved is further explained in Figure D.2. The detailed layout of the ITA region is depicted in Figure D.3.

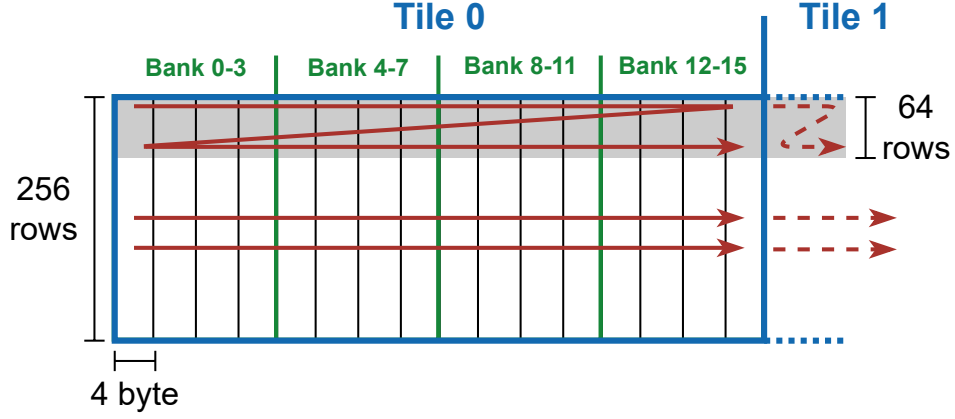


Figure D.2.: MemPool addressing scheme in the default configuration with a banking factor of four and four cores per *tile*. The gray highlighted area represents the sequential region where subsequent addresses first traverse the rows of the same *tile* before continuing to the next *tile*. In contrast, in the interleaved region, the same row of all tiles traversed first, before moving to the next row. As a result, two subsequent rows of the same bank have a distance of $n_{\text{core}} * 4 = 768$ bytes.

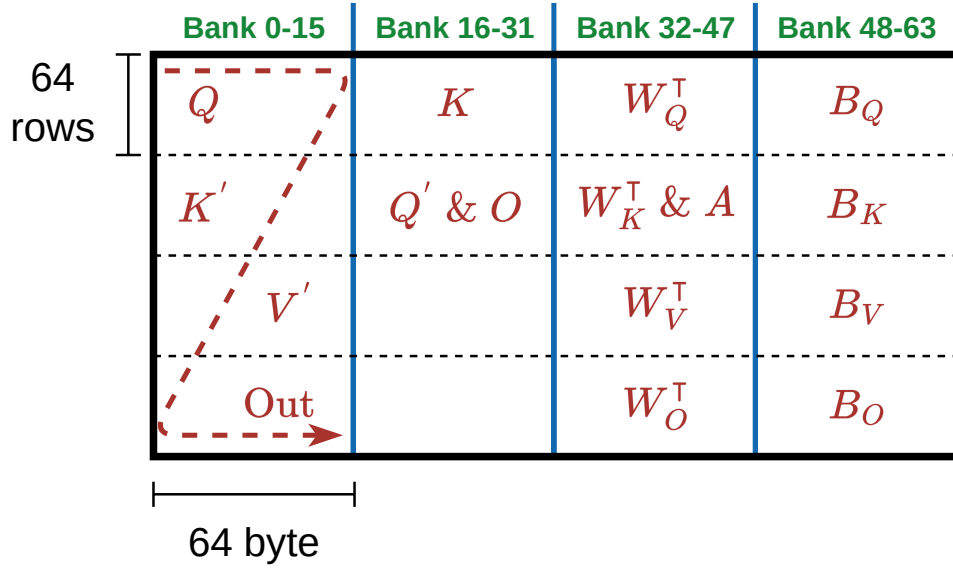


Figure D.3.: Memory layout of the region assigned to one ITA core. The matrices are stored sequentially in memory, each occupying $64 * 64 = 4$ kB. The matrices are arranged in the order shown by the red arrow, where the Out matrix is followed by K in memory and so forth.

Appendix E

Transformer Network

E. Transformer Network

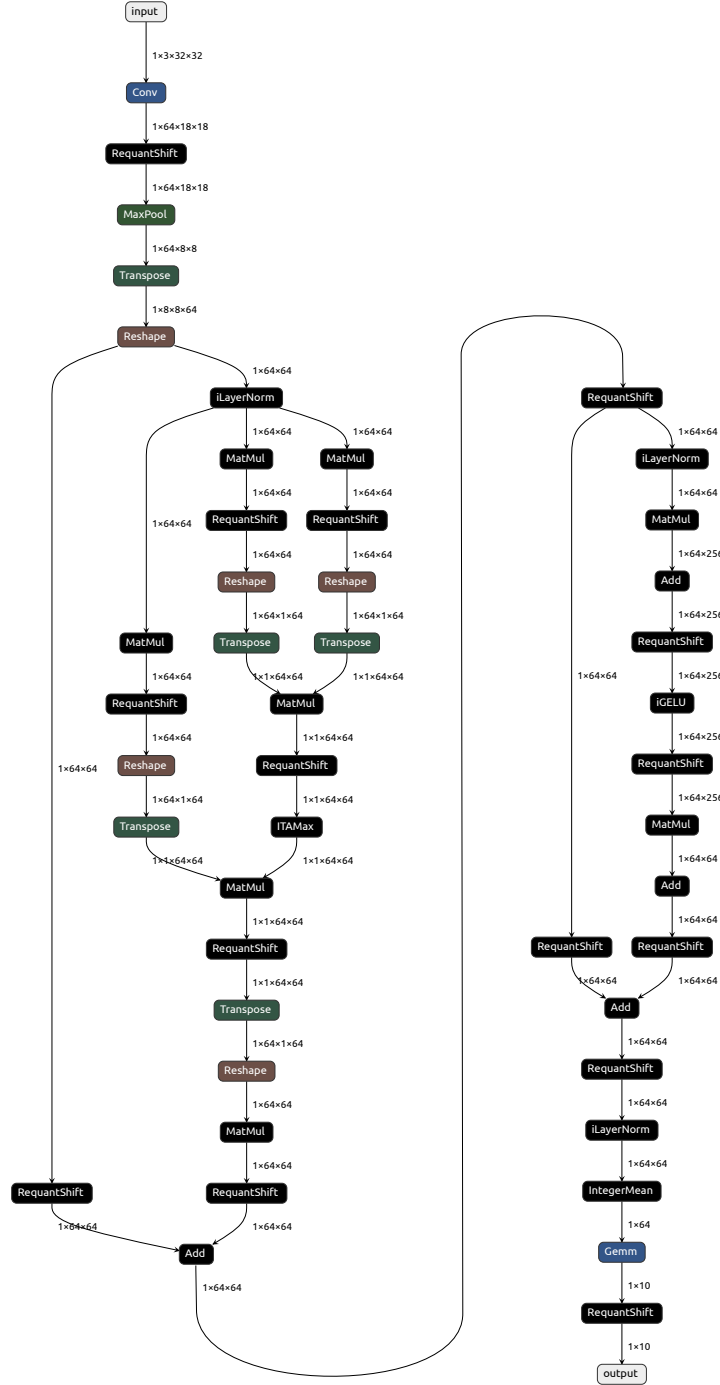


Figure E.1.: ONNX graph of the ICCT network, based on the CCT architecture proposed by [24]. The network has $L = 1$ attention layer with $H = 1$ head and sequence, embedding, and projections dimension $S = E = P = 64$. All re-quantization layers transform the inputs back to 8-bit values.

Glossary

Banshee Banshee is a fast binary simulator supporting up to 618 MIPS for single-core execution by using static binary translation based on the LLVM infrastructure.

Deeploy Deeploy is a tool implementing the *middleware* and *backend* components of an end-to-end deployment toolchain. The tool facilitates fully automated conversion, optimization, and deployment of ONNX graphs to C programmable platforms.

MemPool MemPool is a many-core system targeting image processing applications. It implements 256 RISC-V cores that can access a large, shared L1 memory in at most five cycles.

QuantLab QuantLab is a versatile and modular framework designed for training, comparing, and quantizing neural networks. The framework is based on PyTorch and supports sub-byte and mixed precision quantization.

QuantLib QuantLib is a component of QuantLab containing all quantization related code.

Bibliography

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” 6 2017. [Online]. Available: <http://arxiv.org/abs/1706.03762>
- [2] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-BERT: Integer-only BERT Quantization,” 1 2021. [Online]. Available: <http://arxiv.org/abs/2101.01321>
- [3] OpenAI, “GPT-4 Technical Report,” 3 2023. [Online]. Available: <http://arxiv.org/abs/2303.08774>
- [4] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” 10 2020. [Online]. Available: <http://arxiv.org/abs/2010.11929>
- [5] OpenAI, “DALL·E 2,” 2021. [Online]. Available: <https://openai.com/product/dall-e-2>
- [6] B. Veluri, J. Chan, M. Itani, T. Chen, T. Yoshioka, and S. Gollakota, “Real-Time Target Sound Extraction,” 11 2022. [Online]. Available: <http://arxiv.org/abs/2211.02250>
- [7] G. Islamoglu, M. Scherer, M. Cavalcante, S. Riedel, and L. Benini, “Implementing Architectural Support for Accelerators in MemPool,” 9 2022.
- [8] D. L. Dutta and S. Bharali, “TinyML Meets IoT: A Comprehensive Survey,” 2021.
- [9] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler, “Efficient Transformers: A Survey,” 9 2020. [Online]. Available: <http://arxiv.org/abs/2009.06732>
- [10] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, “A Survey of Quantization Methods for Efficient Neural Network Inference,” 3 2021. [Online]. Available: <http://arxiv.org/abs/2103.13630>

Bibliography

- [11] M. Nagel, M. van Baalen, T. Blankevoort, and M. Welling, “Data-Free Quantization Through Weight Equalization and Bias Correction,” 6 2019. [Online]. Available: <http://arxiv.org/abs/1906.04721>
- [12] Y. Li, R. Gong, X. Tan, Y. Yang, P. Hu, Q. Zhang, F. Yu, W. Wang, and S. Gu, “BRECQ: Pushing the Limit of Post-Training Quantization by Block Reconstruction,” 2 2021. [Online]. Available: <http://arxiv.org/abs/2102.05426>
- [13] J. Choi, Z. Wang, S. Venkataramani, P. I.-J. Chuang, V. Srinivasan, and K. Gopalakrishnan, “PACT: Parameterized Clipping Activation for Quantized Neural Networks,” 5 2018. [Online]. Available: <http://arxiv.org/abs/1805.06085>
- [14] S. R. Jain, A. Gural, M. Wu, and C. H. Dick, “Trained Quantization Thresholds for Accurate and Efficient Fixed-Point Inference of Deep Neural Networks,” 3 2019. [Online]. Available: <http://arxiv.org/abs/1903.08066>
- [15] M. Spallanzani, G. Rutishauser, M. Scherer, A. Burrello, F. Conti, and L. Benini, “QuantLab: a Modular Framework for Training and Deploying Mixed-Precision NNs,” 2022. [Online]. Available: <https://cms.tinyml.org/wp-content/uploads/talks2022/Spallanzani-Matteo-Hardware.pdf>
- [16] S. Riedel, M. Cavalcante, R. Andri, and L. Benini, “MemPool: A Scalable Manycore Architecture with a Low-Latency Shared L1 Memory,” 3 2023. [Online]. Available: <http://arxiv.org/abs/2303.17742>
- [17] S. Mazzola, S. Riedel, M. Cavalcante, L. Benini, and A. Macii, “ISA extensions in the Snitch Processor for Signal Processing,” 2021.
- [18] G. Islamoglu, M. Scherer, G. Paulin, T. Fischer, and L. Benini, “Implementation of an Accelerator for Transformer Networks,” 1 2022.
- [19] S. Riedel, F. Schuiki, P. Scheffler, F. Zaruba, and L. Benini, “Banshee: A Fast LLVM-Based RISC-V Binary Translator ICCAD 2021,” Tech. Rep., 2021. [Online]. Available: http://pulp-platform.org@pulp_platformhttps://www.youtube.com/pulp_platform
- [20] A. Burrello, A. Garofalo, N. Bruschi, G. Tagliavini, D. Rossi, and F. Conti, “DORY: Automatic End-To-End Deployment of Real-World DNNs on Low-Cost IoT MCUs,” *IEEE Transactions on Computers*, vol. 70, no. 8, pp. 1253–1268, 8 2021.
- [21] K. Ueyoshi, I. A. Papistas, P. Houshmand, G. M. Sarda, V. Jain, M. Shi, Q. Zheng, S. Giraldo, P. Vrancx, J. Doevenspeck, D. Bhattacharjee, S. Cosemans, A. Mallik, P. Debacker, D. Verkest, and M. Verhelst, “DIANA: An End-to-End Energy-Efficient Digital and ANALog Hybrid Neural Network SoC,” in *Digest of Technical Papers - IEEE International Solid-State Circuits Conference*, vol. 2022-February. Institute of Electrical and Electronics Engineers Inc., 2022, pp. 256–258.

Bibliography

- [22] M. Scherer, P. Wiese, V. Jung, and L. Macan, “Deeploy: A tool to dump C-Code for Neural Network inference on heterogeneous platforms,” 2023. [Online]. Available: <https://iis-git.ee.ethz.ch/deeploy/deeploy>
- [23] A. Burrello, M. Scherer, M. Zanghieri, F. Conti, and L. Benini, “A Microcontroller is All You Need: Enabling Transformer Execution on Low-Power IoT Endnodes,” in *2021 IEEE International Conference on Omni-Layer Intelligent Systems, COINS 2021*. Institute of Electrical and Electronics Engineers Inc., 8 2021.
- [24] A. Hassani, S. Walton, N. Shah, A. Abuduweili, J. Li, and H. Shi, “Escaping the Big Data Paradigm with Compact Transformers,” 4 2021. [Online]. Available: <http://arxiv.org/abs/2104.05704>
- [25] H. Zhu, B. Chen, and C. Yang, “Understanding Why ViT Trains Badly on Small Datasets: An Intuitive Perspective,” 2 2023. [Online]. Available: <http://arxiv.org/abs/2302.03751>
- [26] B. Keller, R. Venkatesan, S. Dai, S. G. Tell, B. Zimmer, C. Sakr, W. J. Dally, C. T. Gray, and B. Khailany, “A 95.6-TOPS/W Deep Learning Inference Accelerator With Per-Vector Scaled 4-bit Quantization in 5 nm,” *IEEE Journal of Solid-State Circuits*, pp. 1–13, 1 2023. [Online]. Available: https://research.nvidia.com/publication/2022-06_17-956-topsw-deep-learning-inference-accelerator-vector-scaled-4-bit
- [27] F. Tu, Z. Wu, Y. Wang, W. Wu, L. Liu, Y. Hu, S. Wei, and S. Yin, “16.1 MuITCIM: A 28nm 2.24 μ J Token Attention-Token-Bit Hybrid Sparse Digital CIM-Based Accelerator for Multimodal Transformers,” in *Digest of Technical Papers - IEEE International Solid-State Circuits Conference*, vol. 2023-February. Institute of Electrical and Electronics Engineers Inc., 2023, pp. 248–250.
- [28] S. Liu, P. Li, J. Zhang, Y. Wang, H. Zhu, W. Jiang, S. Tang, C. Chen, Q. Liu, and M. Liu, “16.2 A 28nm 53.8TOPS/W 8b Sparse Transformer Accelerator with In-Memory Butterfly Zero Skipper for Unstructured-Pruned NN and CIM-Based Local-Attention-Reusable Engine,” in *Digest of Technical Papers - IEEE International Solid-State Circuits Conference*, vol. 2023-February. Institute of Electrical and Electronics Engineers Inc., 2023, pp. 250–252.
- [29] Y. Wang, Y. Qin, D. Deng, J. Wei, Y. Zhou, Y. Fan, T. Chen, H. Sun, L. Liu, S. Wei, and S. Yin, “A 28nm 27.5TOPS/W Approximate-Computing-Based Transformer Processor with Asymptotic Sparsity Speculating and Out-of-Order Computing,” in *Digest of Technical Papers - IEEE International Solid-State Circuits Conference*, vol. 2022-February. Institute of Electrical and Electronics Engineers Inc., 2022, pp. 464–466.
- [30] R. Krashinsky, O. Giroux, S. Jones, N. Stam, and S. Ramaswamy, “NVIDIA Ampere Architecture In-Depth,” 3 2020. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-ampere-architecture-in-depth/>

Bibliography

- [31] M. Andersch, G. Palmer, R. Krashinsky, N. Stam, V. Mehta, G. Brito, and S. Ramaswamy, “NVIDIA Hopper Architecture In-Depth,” 3 2023. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-hopper-architecture-in-depth/>
- [32] M. Špet’Ko, O. Vysocký, B. Jansík, and L. Říha, “DGX-A100 face to face DGX-2-performance, power and thermal behavior evaluation,” *Energies*, vol. 14, no. 2, 1 2021.
- [33] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” 10 2020. [Online]. Available: <http://arxiv.org/abs/2010.08678>